



Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Sóley: Automated detection of logic vulnerabilities in Ethereum smart contracts using large language models[☆]

Majd Soud^{ID*}, Walteri Nuutinen^{ID}, Grischa Liebel^{ID}

Department of Computer Science, Reykjavik University, Reykjavik, Iceland

ARTICLE INFO

Dataset link: <https://figshare.com/s/efa4967b9eeb9dc6ee96>

Keywords:

Smart contracts
Blockchain
Automation
Software engineering
Vulnerability

ABSTRACT

Context: Modern blockchain, such as Ethereum, supports the deployment and execution of so-called smart contracts, autonomous digital programs with significant value of cryptocurrency. Executing smart contracts requires gas costs paid by users, which define the limits of the contract's execution. Logic vulnerabilities in smart contracts can lead to excessive gas consumption, financial losses, and are often the root cause of high-impact cyberattacks.

Objective: Our objective is threefold: (i) empirically investigate logic vulnerabilities in real-world smart contracts extracted from code changes on GitHub, (ii) introduce Sóley, an automated method for detecting logic vulnerabilities in smart contracts, leveraging Large Language Models (LLMs), and (iii) examine mitigation strategies employed by smart contract developers to address these vulnerabilities in real-world scenarios.

Method: We obtained smart contracts and related code changes from GitHub. To address the first and third objectives, we qualitatively investigated available logic vulnerabilities using an open coding method. We identified these vulnerabilities and their mitigation strategies. For the second objective, we extracted various logic vulnerabilities, focusing on those containing inline assembly fragments. We then applied preprocessing techniques and trained the proposed Sóley model. We evaluated Sóley along with the performance of various LLMs and compared the results with the state-of-the-art baseline on the task of logic vulnerability detection.

Results: Our results include the curation of a large-scale dataset comprising 50,000 Ethereum smart contracts, with a total of 428,569 labeled instances of smart contract vulnerabilities, including 171,180 logic-related vulnerabilities. Our analysis uncovered nine novel logic vulnerabilities, which we used to extend existing taxonomies. Furthermore, we introduced several mitigation strategies extracted from observed developer modifications in real-world scenarios. Experimental results show that Sóley outperforms existing approaches in automatically identifying logic vulnerabilities, achieving a 9% improvement in accuracy and a maximum improvement of 24% in F1-measure over the Baseline. Interestingly, the efficacy of LLMs in this task was evident with minimal feature engineering. Despite the positive results, Sóley struggles to identify certain classes of logic vulnerabilities, which remain for future work.

Conclusion: Early identification of logic vulnerabilities from code changes can provide valuable insights into their detection and mitigation. Recent advancements, such as LLMs, show promise in detecting logic vulnerabilities and contributing to smart contract security and sustainability.

1. Introduction

Smart contracts are self-executing programs and key components of modern blockchain such as Ethereum (Buterin et al., 2014). Smart Contracts have gained widespread attention for their ability to enforce agreements without intermediaries, offering a decentralized and

trustworthy approach (Nakamoto, 2008). The characteristics of smart contracts, including high financial value, immutability, and decentralized nature, have facilitated the growth of decentralized applications (DApps) that offer various functionalities in fields such as gaming, e-commerce, and more (Wood et al., 2014). Smart contracts are typically

[☆] Editor: Yan Cai.

* Corresponding author.

E-mail addresses: majd18@ru.is (M. Soud), niku21@ru.is (W. Nuutinen), grischal@ru.is (G. Liebel).

<https://doi.org/10.1016/j.jss.2025.112406>

Received 5 September 2024; Received in revised form 20 January 2025; Accepted 20 February 2025

Available online 1 March 2025

0164-1212/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

implemented using high-level languages such as Solidity.¹ Their implementation consists of functions that define a sequence of instructions and a set of state variables (Antonopoulos and Wood, 2018).

Ethereum, as the first blockchain supporting smart contracts (Buterin et al., 2014), has garnered significant developer interest, leading to the deployment of over 64 million smart contracts² and a market capitalization exceeding \$300 billion.³ On Ethereum, executing a smart contract requires fees known as “gas”, a unit that measures the consumption of storage and computing resources (Wood et al., 2014). These fees, paid in Ether,⁴ compensate for computing resources used during contract execution.⁵ Smart contracts with logic vulnerabilities are susceptible to severe consequences, including high-impact cyberattacks, execution failures due to excessive gas usage, wasted gas, and significant financial losses (Chaliasos et al., 2024). Identifying logic vulnerabilities is crucial for maintaining Solidity smart contracts as emphasized by Chaliasos et al. (2024). In their study, they highlight the inefficiency of existing smart contract security tools in detecting logic-related vulnerabilities, sanity checks, and logic errors. This challenge is exacerbated by the use of inline assembly, present in approximately 23% of smart contracts (Chaliasos et al., 2022) to add features not supported by Solidity. The complex low-level instructions of inline assembly are only partially supported by existing static analysis tools such as Slither (Feist et al., 2019) and Smartcheck (Tikhomirov et al., 2018a), making these vulnerabilities difficult to detect. Moreover, existing static analysis tools often rely on hardcoded rules that may not capture the details of smart contract interactions, leading to high false positives or missed detections. Their scalability is also a concern, as they may struggle to efficiently analyze larger codebases. Furthermore, the evolving nature of smart contracts and new coding patterns further challenge their effectiveness, highlighting the need for more adaptive detection methods.

Logic vulnerabilities become even more critical when they exist in smart contract functions responsible for transferring Ether out of the contract, as this could potentially prevent the contract owners or users from accessing their money (i.e., Ether). Common logic vulnerabilities include Locked Ether, incorrect equality, access control flaws, and incorrect functionality, which can lead to significant financial losses. A notable example is the Governmental contract (Governmental contract, 2016), where \$2.5 million worth of Ether got locked out. Therefore, ensuring a sustainable and secure development of smart contracts is imperative, and addressing logic vulnerabilities plays a crucial role in achieving this goal. Smart contracts, with their sequential code structure and the interrelation of statements, share similarities with natural language text. Moreover, they often conform to predetermined templates and standards, leading to recognizable patterns and recurring code structures (Clack et al., 2016). Consequently, Large Language Models (LLMs) and other Natural Language Processing (NLP) methods offer the potential for automating the detection of vulnerabilities and generating comments within smart contracts (Hu et al., 2023; Zhao et al., 2024).

Motivated by the advancements achieved by LLMs in conventional software and the inherent characteristics of smart contract code, this paper aims to answer the following research questions (RQs).

- **RQ1 - (Identification):** To what extent do historical code changes reveal logic vulnerabilities in smart contracts?

Motivation: Understanding the relationship between historical code changes and logic vulnerabilities in smart contracts is critical. While previous studies focus primarily on the code itself, they often overlook the impact of code changes and evolution

in revealing these vulnerabilities, which are essential for smart contract security. Our research qualitatively identifies several novel logic-related vulnerabilities while also quantitatively assessing their prevalence. In Table 3, the number of contracts in the dataset that were reported to contain specific types of vulnerabilities are listed. By empirically analyzing historical code changes, we aim to fill a significant gap in the literature and provide evidence that can guide future security practices in smart contracts.

- **RQ2 - (Detection):** How can we automatically detect logic vulnerabilities in smart contracts via LLMs?

Motivation: There is limited research exploring the application of large language models (LLMs) for detecting logic-related vulnerabilities in smart contracts, particularly those involving inline assembly. This research question aims to propose an automated approach based on LLMs to detect these vulnerabilities and explore the performance of well-known LLMs on the same task. It is crucial as it examines the potential of LLMs to uncover complex vulnerabilities that are often overlooked by traditional detection tools.

- **RQ3 - (Mitigation):** What specific strategies do developers employ in their code changes to mitigate potential logic vulnerabilities in smart contracts?

Motivation: This research question aims to qualitatively investigate the mitigation strategies employed by developers to address logic-related vulnerabilities in smart contracts, with a particular emphasis on analyzing real-world code changes. Despite the growing body of knowledge on smart contract security, there remains a significant gap in understanding how developers implement specific modifications to mitigate vulnerabilities in practice. By examining these real-world code changes, we identify best practices in vulnerability mitigation, providing developers with actionable insights. Furthermore, these mitigation strategies support future studies aimed at developing effective guidelines and tools for ensuring robust smart contract security.

With these RQs in mind, our methodology consists of several key steps. First, we gathered instances of smart contract vulnerabilities and their corresponding fixes from real-world Solidity code changes on GitHub. We then investigated the identified logic vulnerabilities using open coding method (Service, 2009), defining the available vulnerabilities and corresponding mitigation strategies. Subsequently, we developed Sóley to effectively detect logic vulnerabilities involving inline assembly fragments using LLMs with minimal feature engineering. Finally, we investigate the performance of various LLMs on the logic vulnerability detection task.

Our work makes the following contributions:

- **Sóley Development and Experiment.** We present Sóley, based on CodeBERTa, to detect logic-related vulnerabilities in smart contracts, particularly those involving inline assembly fragments, with minimal feature engineering. We also explore the performance of well-known LLMs on the same task for various code vulnerabilities in smart contracts.
- **Dataset.** We curated a large dataset of Solidity smart contracts labeled with vulnerability types, locations in the contracts, and other metadata. This dataset contains 50k contracts, 428,569 instances of code vulnerabilities, and 171,180 logic-related vulnerabilities.
- **Analysis of Logic code vulnerabilities and mitigation.** We conduct an in-depth analysis of logic vulnerabilities found in code changes within smart contracts and literature, identifying nine novel vulnerabilities. Moreover, we introduce 15 mitigation strategies used by developers to address logic vulnerabilities.

We have made the models, scripts, and data utilized in this study openly available (SolAI, 2024) to promote research replication and facilitate additional investigations by other researchers in the field.

¹ <https://docs.soliditylang.org/en/v0.8.23/>.

² <https://etherscan.io/contractsVerified>.

³ <https://etherscan.io/>.

⁴ Ethereum cryptocurrency.

⁵ <https://ethereum.org/developers/docs/gas>.

The rest of the paper is organized as follows. In Section 2, we provide background information about smart contracts and the Ethereum blockchain. Our research methodology is outlined in Section 3. Section 4 presents the experiment evaluation and setup, while Section 5 details our findings. Section 6 provides discussions and implications. Section 7 presents the related work. Additionally, we address threats to the validity of our work in Section 8 before concluding in Section 9.

2. Background

2.1. Blockchain and ethereum

Blockchains are transparent, decentralized ledgers of transactions (Zheng et al., 2018). A blockchain consists of a set of blocks, each containing a sequence of transactions that cannot be altered without affecting all subsequent blocks. Ethereum is a blockchain technology featuring a single virtual machine, the Ethereum Virtual Machine (EVM) (Wood et al., 2014). Ethereum provides users with two types of accounts: a smart contract-controlled account or an externally owned account (EOA) (Ethereum Docs, 2023). The latter allows users to initiate a transaction on the EVM to change its states or transfer cryptocurrency (i.e., Ether). Each transaction on the EVM consists of several attributes, such as a transaction hash, sender address, and receiver address. All initiated transactions are added to the blockchain after verification based on cryptographic mechanisms to the blockchain (Chen et al., 2020b).

2.2. Smart contracts and solidity

Smart contracts are self-executing digital agreements, programmed with high-level languages, that require blockchain deployment in order to execute (Szabo, 1996; Wood et al., 2014). They possess three key characteristics: decentralization, immutability, and the ability to handle high financial values. Being decentralized ensures that no single entity controls them, promoting transparency and trust among diverse participants (Atzei et al., 2017a). A smart contract is immutable, which means that it is extremely difficult and expensive to change its code once it has been deployed on the blockchain (Wood et al., 2014). A smart contract facilitates financial transactions, automates payment processing, enforces conditions for fund transfers, eliminates the need for intermediaries, and reduces transaction costs (Destefanis et al., 2018). These features make smart contracts valuable tools for creating secure, transparent, and efficient agreements within decentralized systems.

On Ethereum, smart contracts are written in the Solidity (Solidity Team, 2025) language or Vyper language (Team, 2025), which includes programmable functions and state variables similar to high-level programming languages. Smart contract code is used to force strict rules, commitments, and consequences for parties under several conditions, similar to traditional legal contracts. A smart contract uses parameters within the functions to carry out transactions effectively. For instance, in a smart contract designed to manage token transfers, such as transferring tokens from one user to another, the parameters for this transaction might encompass the sender's address, the recipient's address, and the specific amount of tokens intended for transfer.

Based on the logic implementation of the smart contract, the status of variables changes during the execution of the function. In Ethereum, bytecode is generated from the Solidity contract code. The generated bytecode is then executed by the EVM (Hirai, 2017; Vitalik Buterin, 2014). The generated bytecode is an assembly language that contains low-level instructions called opcodes.⁶ Eventually, each deployed contract is assigned a unique address on the blockchain network. As part of their verification and agreement of the new blocks, each node participating in the blockchain executes the bytecode of the contract.

Users invoke smart contracts deployed on Ethereum by sending a transaction that includes the contract's address, signature, and input parameters for the targeted function. This transaction is directed to the nearest Ethereum node for processing. Smart contracts can manifest as independent applications. For instance, smart contracts may serve as the core or the backend for DApps (Metcalfe et al., 2020).

2.3. Inline assembly in smart contracts

Inline assembly is low-level code that is allowed to be used in Solidity smart contracts (Solidity Team, 2025). There are many reasons to use inline assembly in Solidity, including granting developers fine-grained control over their contracts by implementing libraries or performing operations not available in Solidity (Solidity Team, 2025). Another important reason to use inline assembly with Solidity is to optimize the contract, especially when Solidity's optimizer does not provide efficient code. Inline assembly is written in a language called Yul, which is a readable low-level language. Recently, an empirical study by Chaliasos et al. (2022) showed that around 23% of 49 million contracts contain inline assembly and it is used frequently to add functionality that is not available in Solidity and for gas optimization using specific code patterns. Notably, available static analysis tools that check Solidity contracts (e.g. Slither Feist et al., 2019, Smartcheck Tikhomirov et al., 2018a, and Osiris Torres et al., 2018) only partially support inline assembly fragments.

2.4. Smart contract security

In spite of the benefits that smart contracts provide, they are vulnerable to various security threats (Perez and Zeadally, 2022). In the history of smart contracts, there have been numerous cyber attacks that lead to high financial losses, with large number of smart contracts falling victim to hacking incidents leading to an estimated \$6.45 billion in damages (Chaliasos et al., 2024). The first attack occurred in 2016 when an attack on DAO contracts led to the loss of over 3.6 million Ethers due to a re-entrancy vulnerability (Deconstructing thedao attack, 2016). Due to the increasing frequency of attacks, smart contract security and trustworthiness have gained significant attention from scholars, leading to numerous studies on smart contract vulnerabilities (Atzei et al., 2017b), smart contract analysis (Tikhomirov et al., 2018a), and vulnerability detection methods (Chen et al., 2020a).

2.4.1. Common vulnerabilities in smart contracts

Among the various vulnerabilities that can compromise smart contracts (Soud et al., 2024), the following have been identified and used in the development and evaluation of Sóley:

- **Reentrancy (RE):** A well-known vulnerability in smart contract arises when a function in the contract makes an external call to another untrusted contract before resolving the previous state, which allows the untrusted contract to make recursive calls back into the original function, resulting in draining the gas from the contract (Soud et al., 2024; Tikhomirov et al., 2018a).
- **Recursive Calls in Loops (CLP):** Recursive calls within loops, particularly when combined with inline assembly, can result in excessive gas consumption and denial of service, as the contract may exceed the gas limit (Atzei et al., 2017a).
- **Incorrect Low-Level Calls (LLC):** low-level function calls, such as 'call', 'delegatecall', or 'staticcall', leading to failed executions or unintended behaviors if not checked (Durieux et al., 2020).
- **Locked Ether (LE):** the contract logic only locking Ether balance all the time because of its inability to access the external library contract to transfer Ether which increases the risk of funds becoming permanently inaccessible (Soud et al., 2024).

⁶ <https://ethereum.org/en/developers/docs/evm/opcodes/>.

- **Incorrect Equality Check (IE):** Faulty equality checks in conditional statements, especially when involving inline assembly, can result in incorrect logic execution, leading to security breaches (Tikhomirov et al., 2018b).

The biggest losses of ether have been caused by reentrancies. Vulnerabilities such as the reentrancy attack can be caused by different parts of the code interacting with each other or other contracts, complicating the detection process. In the analysis report from the tools, the affected lines could contain either a large part of the code making it difficult for the contract developer to find where the issue is, or a very short section of the contract leaving the developer to solve what in the context of the section makes it vulnerable. Low-level calls can be used safely for example, but this depends on what they are interacting with and whether the calls are checked, much like the state changes with reentrancies.

Durieux et al. (2020) conducted an empirical review of smart contract analysis tools, analyzing more than 45 000 smart contracts with multiple tools and comparing the findings. Their comparisons show very significant differences in the detection rates between the different tools; not every tool is designed to detect every vulnerability, while others also give high false positive rates for certain vulnerabilities. *Mythril* is shown to have the best overall performance, detecting 27% of the 115 vulnerabilities tested, but the tool *Slither* outperforms it in detecting reentrancies, for example. However, *Slither* did not detect any of the arithmetic vulnerabilities, while *Mythril* detected 68% of them.

This kind of complexity in detecting different types of vulnerability leads us to look at using a deep learning model for the detection. Transformer models specifically are designed to consider the context of each word in a sentence; they were originally designed for machine translation and natural language processing. They have also shown to perform well with programming languages, with models trained specifically on them.

2.5. Large Language Models (LLMs)

Transformers enabled the development of LLMs such as BERT (Bidirectional Encoder Representations from Transformers) (Devlin et al., 2018) and GPT (Generative Pre-trained Transformer) (Radford and Narasimhan, 2018), which achieved remarkable performance across a wide range of NLP tasks, including text classification, sentiment analysis, language translation, and more. Based on the success of Transformers in NLP, scholars have expanded their application to programming languages. For instance, CodeBERT and CodeBERTa (Feng et al., 2020; Hugging Face, 2024) can capture semantic relationships between code and plain language. They offer generic code representations that enable various code automation tasks, providing developers with more resources for comprehending and producing code. BERT was pre-trained on a large corpus of English text, including Wikipedia. It was trained using two primary tasks: masked language modeling and next sentence prediction, where the model predicts if two given sentences follow each other in the text. Therefore, BERT connects the context of the words in sentences. DistilBERT is a distilled version of BERT (Sanh et al., 2019). The model's design involves the removal of token-type embeddings contributing to a more efficient and faster processing capability. DistilBERT also simplifies the pre-training process by eliminating the Next Sentence Prediction (NSP) mechanism, while retaining the essential Masked Language Modeling (MLM) mechanism. GPT-4 (OpenAI, 2023), developed by OpenAI, was trained on a diverse dataset that includes 8 million web pages in the English language. GPT-4 leverages advanced transformer architecture to understand and generate human-like text. T5 (Raffel et al., 2020), developed by Google, can perform various natural language processing tasks, such as text translation with high accuracy. By converting all tasks into a text-to-text format, T5 leverages a unified framework that simplifies training and improves performance across different NLP tasks. In summary,

Transformer-based architectures, have enabled applications to analyze and detect code vulnerabilities using their deep contextual understanding and powerful NLP capabilities. Feng et al. (2020) compare the performance of models on code-to-documentation generation, classified by the type of data the models were pre-trained on. The models that were pre-trained on programming languages outperform the generic transformer-implementation, showing a BLEU-4 score increase of 8.7% to 20.5%. However, the models can face difficulty when code segments with different purposes look similar, as in the case with smart contract vulnerabilities. Some vulnerability types can have similar looking syntax, but a different function depending on the context and relevant code. If we include a large amount of samples where the vulnerability contains an equality operator, the model might assume any equality operation is vulnerable. This is expanded on in Section 5.2.

3. Method

In this section, we outline our research method. We begin by presenting the process of collecting and cleaning the smart contracts and code changes from GitHub. Afterward, we detail our qualitative approach to investigate the available logic vulnerabilities and corresponding mitigation strategies. Finally, we describe the code pre-processing, training, and evaluation phases for our proposed Sóley — automated smart contract logic vulnerability detection.

3.1. Data collection

In this work, we collect an extensive dataset of real-life smart contract source code and code changes available on Open Source Software (OSS) projects hosted on GitHub using AutoMESC (Soud et al., 2023).

We selected GitHub, as it is the largest host of source code globally (Wartschinski et al., 2022). Moreover, smart contracts on GitHub are presented in their solidity source code form, which includes additional essential elements such as code changes, reviews, code vulnerabilities, commits, and fixes. These components significantly enhance the comprehensibility of the code during our analysis. GitHub as a version control system that is centered around commits. A commit that fixes a code vulnerability can be viewed as a fix, comprising two crucial pairs of source code: the initial version with code vulnerabilities and the subsequent updated and, ideally, fixed version as highlighted by (Zhou and Sharma, 2017). Analyzing these commit differences (diffs) offers valuable insights into fixing the smart contract.

Fig. 1 shows our data collection process (left side). First, we collected contracts specifically written in Solidity (identified by the .sol extension) from various projects on GitHub, including the official Ethereum repository (Ethereum, 2024) and Smartbugs (Durieux et al., 2025), resulting in a corpus of Solidity smart contracts. Notably, these contracts include real, publicly deployed versions with blockchain addresses, reflecting their active use on the Ethereum blockchain.

We extracted code changes indicating code vulnerabilities or fixed code by gathering the commit differences (diffs) for each smart contract (i.e., each .sol file), where available. However, some of these diffs were found only in comments and not in the actual contract code. We filtered out these comment-based changes, focusing solely on modifications made within the Solidity code. Our data collection and initial filtering were conducted through an automated pipeline utilizing AutoMESC (Soud et al., 2023).

3.2. Data cleaning and pre-processing

We performed data cleaning by processing the collected data from the previous step to identify potentially duplicate smart contracts and code changes. In the duplicate filtering stage, where we retained contract URLs and their source codes, we proceeded to compare Solidity contract names and URIs. Through this process, we detected possible duplicates and removed them. Then we compiled the resulting

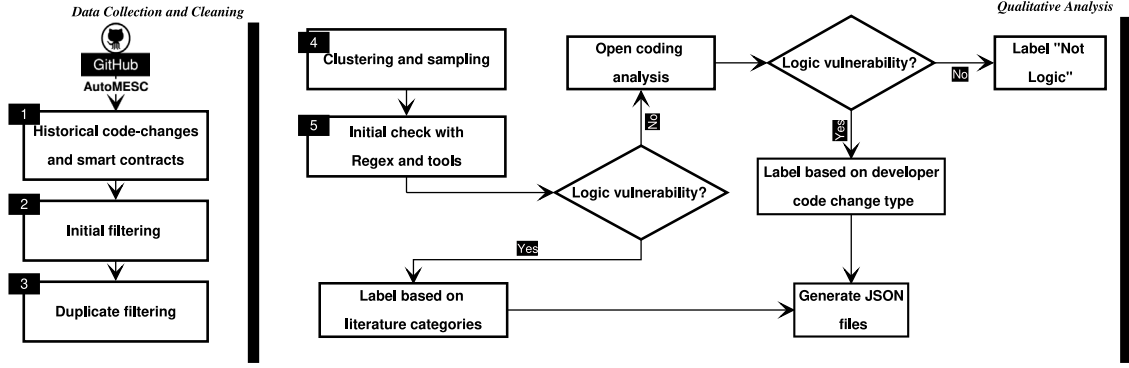


Fig. 1. Overview of our data collection and qualitative analysis. Steps 1 to 3 depict data collection and cleaning, while all remaining steps depict the analysis. The two parts are disconnected as they could theoretically be conducted independently, e.g., for replication.

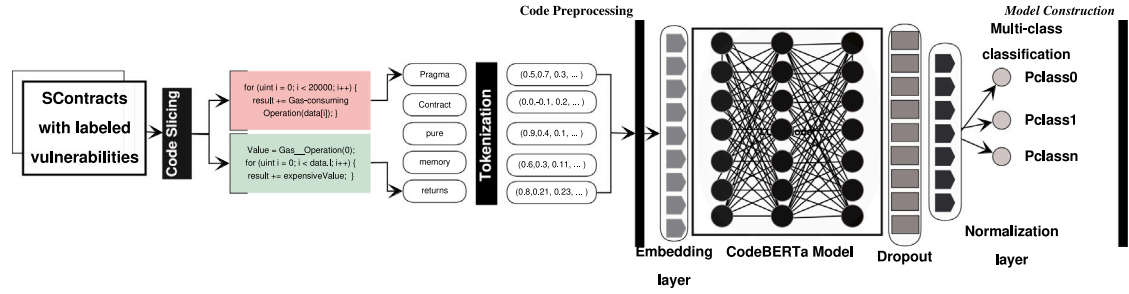


Fig. 2. Overview of our code preprocessing and model construction.

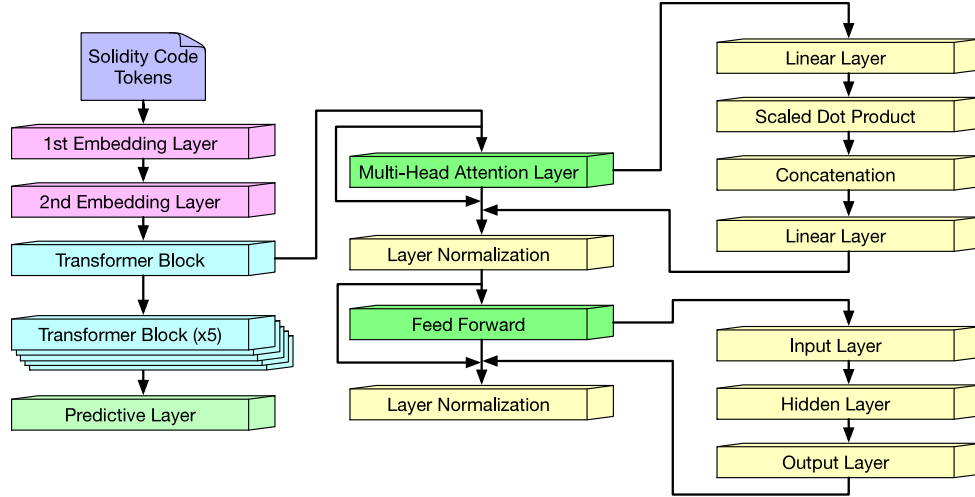


Fig. 3. The architecture of BERT-based models. In this case the model uses 6 transformer blocks, and each block includes some dropout layers to reduce overfitting. Dropout layers set some input values to 0, so the model does not learn to recognize sample-specific features.

contracts to remove the non-functional contracts using the Solidity compiler (Solidity Team, 2025). Subsequently, we assigned unique hashes to distinct contracts and linked code change pairs with their corresponding contracts via AutoMESC. This approach yielded a corpus comprising a set of unique smart contracts, metadata related to these contracts, as well as the source code of the contracts and their associated code changes when available. The resulting dataset served as the input for our qualitative analysis. Finally, smart contracts containing inline assembly were identified using carefully designed regular expressions and corresponding static analysis tools, as detailed in Section 3.4. The regular expressions used for this identification can be found in Appendix.

3.3. Quantitative overview of the collected dataset

Our dataset consists of smart contracts collected from GitHub, along with smart contracts with code changes obtained from GitHub code reviews and diffs. Table 1 presents the quantitative analysis of the collected smart contracts and code changes before and after filtering and duplicate removal.

The total number of collected contracts written in Solidity is 54,652. Of these, 2194 contracts were non-functional or duplicates. As a result, we obtained 52,458 functional and unique contracts. Among the smart contracts with at least one code change, there were a total of 13,000 contracts, including duplicates, and approximately 6000 unique contracts. The total number of unique code changes is around 12,000.

Table 1
Quantitative overview of the collected dataset.

Description	
Total number of collected contracts	54 652
Total number of unique functional contracts	52 458
Total number of non-functional and duplicate contracts	2194
Total number of unique contracts with code changes	6363
Total number of lines for all contracts with code changes	1 544 321
Total number of vocabulary	305 377
Average number of lines	242.703
Average number of vocabulary	47.993
Min number of lines	7
Max number of lines	7829
Min number of vocabulary	30
Max number of vocabulary	24 594

Table 2
Clustering of smart contract code changes.

Cluster	Total
Standard compliance	1099
Storage	1005
Pragma changes	306
Contract Oracle	147
Assembly	145
Upgradability	48
Random	1805
Testing	1445

3.4. Qualitative analysis approach

Fig. 1 shows the steps we followed to perform our qualitative analysis. Our proposed qualitative analysis approach starts with clustering the code changes based on general themes and then selecting random samples from each cluster for analysis, as we describe next.

We carefully examined a random set of 1000 unique code changes with varying vocabulary sizes and in various contracts in the first round to understand the types of changes developers typically make to their contracts. After that, we automatically clustered the code changes based on common characteristics (i.e., frequent keywords) into eight clusters using a Python script. The resulting clusters are described in Table 2.

We then selected a random 10% of code changes from each cluster, except for the Upgradability and Pragma Changes clusters. We selected all 48 code changes from the Upgradability cluster considering its small size. Additionally, we selected 48 code changes from the Pragma Changes cluster, given that the initial analysis revealed no explicit logic vulnerabilities or mitigation strategies in that cluster.

After sampling and clustering steps, we started the analysis of the code changes to obtain the types of logic vulnerabilities that exist in these changes, as shown in Fig. 1. First, we checked if the available code vulnerabilities corresponded to any of the defined logic vulnerabilities in the literature, assigning labels accordingly. We extracted the defined logic vulnerabilities from Sun et al. (2024), Soud et al. (2024), Zhang et al. (2020) and Zhang et al. (2023) and utilized static analysis tools such as Slither (Feist et al., 2019) and Smartcheck (Tikhomirov et al., 2018a), along with regular expressions to identify logic-related vulnerabilities. These tools have been validated and employed in recent studies to analyze and consolidate smart contract datasets (Zheng et al., 2024; Durieux et al., 2025). The generated labels were saved in a JSON file for each contract. Two reviewers—a doctoral student expert in smart contracts and software security, and a master's degree student expert in smart contracts and machine learning—examined the regular expressions in three rounds. If no logic vulnerability was identified in the code change, we employed an open coding method for analysis. The reviewers manually inspected the code changes, related contracts for context, and any developer comments. In cases where the code change belonged to a new logic vulnerability not identified by the

literature, we relied on developer comments or the type of the code change to assign it a name. Otherwise, we labeled it as a non-logic-related vulnerability. The qualitative analysis resulted in labeled JSON files containing contracts, code changes, and their corresponding logic vulnerabilities, enabling us to extract categories of logic vulnerabilities and their mitigation strategies.

To support our automated logic vulnerability detection, a substantial amount of data is needed. To achieve this, we utilized regular expressions on the contracts we had, in addition to employing various analysis tools mentioned earlier to identify known logic vulnerabilities in smart contracts. If the tools' output aligns with the output from the regular expressions, we automatically labeled the vulnerability and generated JSON files. Each JSON file contains information about the detected lines with code vulnerabilities, including the vulnerability type, contract filename, and some metadata. In the manually labeled JSON file, we included an additional attribute that provides an explanation of why the detected code change is considered a code vulnerability. This explanation facilitates replication and verification by the research community.

3.5. Code preprocessing and granularity level selection

Solidity contracts, despite sharing identical tokens, may vary in cleanliness or susceptibility to code vulnerabilities. This highlights the need for a detailed analysis of individual lines of code, rather than relying solely on a top-down analysis of entire contracts. Moreover, previous research indicates that processing entire files for classification has limitations in effectively identifying code vulnerabilities (Morrison et al., 2015).

Therefore, our study adopts a fine granularity approach similar to that of Wartschinski et al. (2022) by examining code tokens⁷ responsible for logic vulnerabilities and their contextual usage by considering sequences of related code tokens. This approach enables us to precisely pinpoint the location of code vulnerabilities within sequential code, considering the interdependence of statements. Considering a specific token as a code vulnerability may result in many false positives. Instead, we assume that a token signifies a code vulnerability when utilized in a particular way. Therefore, we consider the surrounding tokens and lines, based on the lines available in the generated JSON files.

3.6. Code slicing and tokenization

After determining our study's granularity level, we proceeded with the code-slicing phase (Weiser, 1984) as shown in Fig. 2. This phase involved partitioning the code into two categories: negative slices containing code lines demonstrating logic vulnerabilities, and positive slices containing code lines without such vulnerabilities. We iterated through each contract and extracted the negative slices along with the corresponding type of logic vulnerability and the positive slices. To extract an inline assembly segment specifically, we used regular expressions. To extract any inline assembly, we search for the *assembly* identifier, and then match the number of open and closed curly brackets. This was done in Python, with a loop that reads the lines of code looking for symbols and keeps track of them. To help labeling the types of vulnerabilities directly from inline assembly, a list of the expressions is shown in Appendix. The expressions look for specific operations, that match with a vulnerability type in vulnerable contracts. The operations themselves are not necessarily always vulnerable. The tokenizer utilized in our study is a byte-level BPE tokenizer trained on the corpus using Hugging Face tokenizers.⁸ Sóley operates on the full

⁷ Fundamental building blocks of the code, such as keywords, identifiers, operators, and literals.

⁸ <https://huggingface.co/huggingface/CodeBERTa-small-v1>.

source code of the sliced smart contract, preserving the code exactly as it is. To transform a slice of code into a numerical representation, the tokenizer first splits the code into a list of Solidity code tokens, including keywords (e.g., “contract”, “function”), identifiers (e.g., variable names), operators (e.g., “+”, “.”), and other language-specific constructs. The tokenizer provides a dedicated lexical scanner for various source codes, including Go, Python, and Java. For instance, it splits a line such as “function transfer(address _to, uint256 _value) public returns (bool success)” into the tokens: “function”, “transfer”, “(”, “address”, “_to”, “,”, “uint256”, “_value”, “)”, “public”, “returns”, “(”, “bool”, “success”, “)”, “,”, “.”. Each of these tokens is then embedded, i.e., represented by a numeric vector. Consequently, a full section of code is transformed into a series of numbers.

3.7. Model selection and construction

Source code inherently consists of sequential data, where the effect of each statement depends on surrounding instructions. Hence, we need a model capable of learning features associated with code vulnerabilities from sequences of code tokens. Machine learning-based models, especially transformers, excel at representing the sequential nature of code while capturing its semantics. These models are adept at tasks similar to our task, as they can effectively model code and have been successfully utilized in similar contexts. Previously, we experimented with models based on different architectures such as GPT, T5, and BERT. We tested the models in detecting whether a smart contract was buggy or not by training them on a dataset of contracts containing bugs and their fixed counterparts. This data was gathered by Soud et al. (2024) from revised contracts, and thus the buggy code and its replacement could be extracted from the file differences. We used these code segments as input when training the model. We also compared the training times and resource requirements if the model were to be trained locally. In our testing, BERT, RoBERTa, and similar models tended to reach good accuracy within a few training epochs, averaging around 90% accuracy. Therefore, we selected CodeBERTa (Hugging Face, 2024) as the model for this work, as the performance appeared to be similar while the training times were shorter. The chosen model consists of 6 layers of transformer blocks and 84 million parameters. It is a RoBERTa-like transformer model that shares the same number of layers and heads as DistilBERT. The model was initialized with default settings and trained from scratch on the full corpus. The tokenizer is a byte-level BPE tokenizer, originally trained on a corpus of code from the CodeSearchNet dataset. Compared to the standard RoBERTa tokenizer, which is trained on a natural language, a tokenizer trained specifically on code encodes the code more efficiently leading to shorter token lengths. This means the tokenized code should contain more detailed information for the model make connections from.

The selected CodeBERTa model architecture includes a series of layers and components designed to process input data effectively. The architecture is depicted in Fig. 3. The model begins with embedding layers that map input tokens to high-dimensional representations. The first embedding layer has a vocabulary size of 52,000 and outputs 768-dimensional embeddings, while the second embedding layer has a vocabulary size of 514 and outputs 768-dimensional embeddings.

Following the embedding layers, dropout layers are applied to prevent overfitting by randomly setting a fraction of input units to zero during training. Layer-normalization layers are then applied to normalize the outputs of the previous layers, ensuring stable training. The model also includes multiple convolutional layers which perform convolution operations on the input embeddings. These convolutional layers are interleaved with dropout layers and layer normalization layers to improve the model’s ability to capture complex patterns in the data.

An activation function called NewGELUActivation is applied after every second convolutional layer. After the convolutional layers, additional dropout and layer normalization layers are applied to further regularize the model. Finally, it has a linear layer that maps the 768-dimensional input to a 6-dimensional output for multi-class classification.

3.8. Model training

From the preprocessed data, we randomly selected 1000 samples of five smart contract vulnerability types, all containing inline assembly fragments. These vulnerability types were chosen based on their prevalence and significance in existing literature (Chaliasos et al., 2024; Soud et al., 2024), to provide a diverse representation of smart contract vulnerabilities. However, for a sixth logic vulnerability type, we could only select 592 samples because these were the only ones with inline assembly, while excluding those without. Inline assembly is categorized as logic-related by Zhang et al. (2020), so we included all vulnerabilities with inline assembly fragments to address the gap where existing tools fail to detect these vulnerabilities (see Section 7).

Our selection comprises diverse vulnerability types, some easily detectable while others require nuanced comprehension and deep semantic understanding, ensuring the comprehensiveness of our selection. The diversity in our selection highlights the critical importance of addressing a broad range of vulnerabilities for thorough coverage.

To ensure consistency in the training process and assess the accuracy of our model and baseline models, we maintained identical hyperparameters throughout. The length of text sequences was limited to 512 tokens, a common choice in NLP tasks that balances the need to capture sufficient contextual information while remaining manageable within our memory constraints (Brown, 2020). A batch size of 10 was selected, which, while limiting training speed, was necessary to accommodate the constraints when training larger models on local machines (Goyal, 2017), such as ours with 16 GB of RAM and 4 GB of VRAM. Smaller batch sizes can also provide more stable gradient estimates, aiding in model convergence. These parameters are interrelated, as they dictate the memory demands for each batch. While a higher batch size can expedite training, it also necessitates more memory.

For optimization, we utilized the AdamW optimizer with a learning rate of $5e-5$ and epsilon of $1e-8$, as prescribed in the original GPT-2 paper. The learning rate was initially set to larger steps to facilitate quicker convergence, then adjusted to a smaller value as training progressed, allowing for a more refined learning process. This iterative approach enabled better navigation of the loss landscape, reducing the risk of overshooting optimal solutions.

All models underwent 10 epochs of training, with evaluation statistics recorded at each iteration. Training larger models locally presents challenges due to their substantial memory requirements. However, we mitigated this issue by utilizing smaller models with reduced batch sizes, enabling training on our local machine. Although training times were longer compared to GPU clusters, attempting to initialize larger models on GPUs risked memory constraints. While decreasing sequence length and batch sizes helped mitigate these challenges, it also resulted in longer training times. For training, we split the data into a 75/25 ratio for training and evaluation, following the default setting of the ‘train_test_split’ function from scikit-learn.

4. Experiment evaluation

In this section, we describe our experiment and evaluation measures. Additionally, we present the selected baseline.

4.1. Metrics and evaluation

We trained each of the models on the same dataset, with the same settings and hyperparameters, and then ran an evaluation benchmark from the scikit-learn⁹ package. For the evaluation, we used precision, recall, and F1-measure, as well as an accuracy calculation, all from the scikit-learn package. Accuracy was determined by accumulating

⁹ <https://scikit-learn.org/stable/>.

Table 3
Distribution of selected vulnerabilities across contracts.

Vulnerability	Number of contracts
Reentrancy	17,625
Calls Loop	3019
Uninitialized Local	3462
Low-level Calls	5555
Locked Ether	4563
Incorrect Equality	2611

correct predictions throughout the evaluation process and computing the average accuracy score at its conclusion.

$$\text{Accuracy} = \frac{\text{Number of Correct Prediction}}{\text{Number of Samples}}$$

The precision value describes the model's ability to not falsely label negative samples as positive. Recall, on the other hand, measures the model's ability to accurately capture all positive samples.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Finally, the F1-measure represents the harmonic mean of precision and recall. This metric provides a balance between precision and recall, considering both false positives and false negatives.

The accuracy in this case reflects the overall performance over all the samples, while precision, recall, and F1-measure were evaluated for each type of vulnerability separately. This approach highlights whether the model struggles in detecting specific types of logic vulnerabilities.

4.2. Baseline selection

Baseline 1: We employ the base model used by Sun et al. (2023) for the task of detecting six types of smart contract vulnerabilities, including Reentrancy and flow vulnerabilities. The selected model leverages various implementations of the BERT architecture. This model has been widely recognized as a ubiquitous baseline in NLP experiments for its widespread use and robust performance, as noted by Rogers et al. (2020). We adopt this baseline model for its application in detecting vulnerabilities in smart contracts. Following the description in the paper, we utilized the base BERT architecture, as the original code was not provided.

5. Results

This section presents our research findings and addresses the proposed research questions. We begin by analyzing logic-related vulnerabilities in real-world smart contract code changes. Next, we evaluate S ley's automated detection capabilities across various vulnerability types and compare the results with other well-known LLMs performing the same task. Finally, we summarize the strategies developers employ to mitigate potential logic vulnerabilities.

5.1. Answers to (RQ1): Logic vulnerability types

To address the RQ1, regarding logic vulnerabilities in our dataset, we begin by conducting a qualitative analysis of the logic-related vulnerabilities identified by the selected tools in our dataset. Overall, the total number of vulnerabilities in our dataset is 428,568. Of these, 171,180 are logic-related code vulnerabilities.

We present a comprehensive analysis of the vulnerabilities available in our dataset in Appendix. Additionally, a quantitative summary of a selected subset of vulnerabilities across contracts is provided in Table 3, while the remaining vulnerabilities can be found in Appendix.

Before starting our manual analysis, we reviewed publications on logic vulnerabilities in smart contracts to identify known vulnerabilities, as summarized in Table 4. We identified 17 logic vulnerabilities from the literature. We present the list of identified logic vulnerabilities along with their definitions and corresponding references in Table 5.

We then proceeded to manually label 614 code changes sampled from the clusters we have as mentioned in the method Section 3, from which we were able to define nine logic vulnerabilities. In the following section, we explain each defined logic vulnerability. While certain vulnerabilities may manifest in various forms within smart contract code, due to space constraints in this paper, we present code examples when needed. Additional examples and various forms of these vulnerabilities can be found in our dataset, where the full contracts and code changes are available.

Listing 1: Bypassing Solidity access control via inline assembly

```
function setSecret(uint256 _secret) public {
    require(isAdmin[msg.sender], "Only admin
        can set secret");

    // Some code for setting secret here
}

function setSecretWithAssembly(uint256 _secret)
    public {
    assembly {
        // Load the isAdmin mapping slot
        let slot := sload(isAdmin.slot)

        // Logic-related vulnerability
        // Load the value from the mapping using slot
        and msg.sender
        let isAdminValue := sload(add(slot,
            mul(and(isZero(shr(96,
                calldataload(0))), 0xffffffff),
                0x20))) }}}
```

5.1.1. Bypassing solidity access control via inline assembly

Inline assembly in Solidity allows developers to directly integrate assembly code within their Solidity contracts to effectively execute certain operations with less gas cost and added flexibility (Chaliasos et al., 2022) (see the Background Section 2). However, one significant risk is that inline assembly can bypass Solidity's access control mechanisms, affecting the logic and flow of the contract. Solidity's access control mechanisms typically depend on modifiers or conditional statements in Solidity code to enforce restrictions on function calls based on certain conditions (Solidity Team, 2025). Attackers or unauthorized users could potentially use inline assembly logic to gain unauthorized access to restricted function logic, even if they do not meet the specified conditions for access according to the Solidity code. In the following, we illustrate this vulnerability with example Listing 1. In the example, we have a "setSecret" function in Solidity that utilizes an access control mechanism, specifically employing a "require" statement to ensure that only admin users can set a secret. No other user should have the privilege to set the secret, making the admin the sole entity with this capability. However, we also have a "setSecretWithAssembly" function that bypasses the implemented access control mechanism. This is achieved by directly accessing the "isAdmin" mapping slot using inline assembly. Consequently, an unauthorized user can call "setSecretWithAssembly" and set the secret without being checked against the admin role. This leads to the unintended usage of the logic implemented in the contract and eventually compromises its security.

Table 4

Publications discussing or identifying logic vulnerabilities in smart contracts.

Papers	Venue	Logic vulnerabilities
Sun et al. (2024)	ICSE	Defined nine scenarios of known logic vulnerabilities in smart contracts.
Soud et al. (2024)	EMSE	Identified four logic vulnerabilities in smart contracts.
Chaliasos et al. (2024)	ICSE	Tested existing smart contract tools for logic vulnerabilities, errors, and sanity checks.
Zhang et al. (2020)	ICSME	Identified eight logic vulnerabilities in smart contracts and categorized them into four categories.

Table 5

Identified logic vulnerabilities in the literature, along with their definitions and corresponding references.

Vulnerability	Description	Ref.
Unauthorized transfer	A smart contract allows assets to be transferred without proper authorization.	Sun et al. (2024)
Risky first deposit	The first deposit into a contract sets critical parameters that can be exploited by the first user, resulting in an unfair advantage.	Sun et al. (2024)
Price manipulation	Attackers manipulate asset prices by exploiting the liquidity pool algorithms in decentralized exchanges using Automated Market Makers (AMMs).	Sun et al. (2024)
Front running	An attacker observes a pending transaction and places a similar transaction with higher fees to be processed first, which results in financial gain at the expense of the original transaction.	Sun et al. (2024)
Wrong interest rate order	Interest rates are calculated or applied in an incorrect sequence, leading to misallocation of interest payments.	Sun et al. (2024)
Wrong checkpoint order	Errors in the sequence of recorded checkpoints within a contract, which results in incorrect contract behavior or mismanagement of funds.	Sun et al. (2024)
Slippage	The difference between the expected price of a trade and the actual price executed, often causing losses for traders due to low liquidity.	Sun et al. (2024)
Greedy contract	A smart contract that consumes excessive resources (e.g., gas) during function calls and locks user funds (i.e., Locked Ether).	Soud et al. (2024)
DoS by external contract	A smart contract relies on an external contract that can be manipulated or fails to respond.	Soud et al. (2024)
Call to the unknown	A smart contract calls a function on an unknown or user-specified address without proper validation, potentially executing malicious code.	Soud et al. (2024) , Zhang et al. (2020)
Transaction Order Dependency (TOD)	The outcome of a contract depends on the order of transactions within a block, allowing attackers to gain an unfair advantage by influencing transaction order.	Soud et al. (2024) , Zhang et al. (2020)
Returning results using assembly in the constructor	Using low-level assembly language in a constructor to return data, bypassing typical restrictions and potentially leading to complex logic.	Zhang et al. (2020)
Specify function variable as any type	Using a generic or ambiguous data type for function parameters, leading to unexpected logic behaviors.	Zhang et al. (2020)
DoS by gas limit	Exploiting the gas limit by creating situations where the required gas exceeds the block gas limit.	Zhang et al. (2020)
DoS by complex feedback function	Invoking a function involving complex computations or recursive calls that consume excessive gas.	Zhang et al. (2020)
DoS by non-existent function	Calling a function that does not exist, causing the transaction to fail and revert.	Zhang et al. (2020)
Storage overlap attack	Different variables or mappings share the same storage slot due to incorrect definitions, allowing manipulation of one variable by changing another.	Zhang et al. (2020)

5.1.2. State manipulation via inline assembly

This vulnerability involves manipulating state variables within inline assembly, particularly under conditions that may not be immediately apparent. Such actions can lead to state inconsistencies within the logic of the contracts. These inconsistencies introduce uncertainty into the contract functionality and logic flaws. Consequently, developers and auditors may face difficulties in fully assessing the contract's logic and behavior. In Listing 2, the “conditionalManipulation” function employs inline assembly to modify the “manipulatedValue” only if the input _value exceeds a specific threshold, in this instance, 1000. This manipulation has the potential to disrupt the logic of the “withdraw” function, which verifies “manipulatedValue” before permitting withdrawals. If “manipulatedValue” is altered to an unexpected value, the “withdraw” function reverts, which shows how state manipulation can alter the logic of the contract.

Listing 2: State Manipulation via Inline Assembly

```
function conditionalManipulation(uint256 _value)
    public {
    assembly {
        // Conditionally store value in
        // manipulatedValue's slot
        if gt(_value, 1000) {
            sstore(manipulatedValue.slot, _value)
        }
        emit Manipulation(_value);
    }

    function withdraw(uint256 _amount) public {
        require(balances[msg.sender] >= _amount,
            "Insufficient balance");
        require(totalSupply >= _amount, "Insufficient
            total supply");

        // Check if manipulatedValue has been set to an
        // unexpected value
        if (manipulatedValue > 1000) {
            revert("State manipulated, cannot
                proceed");
        }
        balances[msg.sender] -= _amount;
        totalSupply -= _amount;
        payable(msg.sender).transfer(_amount);
        emit Withdraw(msg.sender, _amount);
    }
}
```

5.1.3. Incomplete standard function implementation

This vulnerability occurs when a smart contract fails to implement all the required functions of a standard or modifies them, resulting in an incomplete adherence to the standard (Community, 2024). As a consequence, the contract's logic is compromised, potentially leading to unexpected behavior when interacting with other contracts that assume full compliance with the standard.

Consider an ERC-20 (Community, 2024) token contract that does not properly implement the “transferFrom” function, leading to a vulnerability that can cause unexpected behavior when transferring tokens between addresses. Therefore, it can potentially compromise the logic of other smart contracts or dApps built on top of the ERC-20 token standard. In this case, the logic of the ERC-20 token contract itself is compromised due to the improper implementation of the “transferFrom” function. For instance, if the “transferFrom” function fails to properly deduct the transferred token amount from the sender's balance or update the recipient's balance, it can lead to inconsistencies in token balances and potentially allow unauthorized token transfers. As a result, the contract's core logic for managing token transfers and maintaining token balances may become unreliable, impacting the overall functionality and security of the token contract.

5.1.4. Incorrect validation of oracle data

In blockchain, oracles serve as crucial links between off-chain data sources and on-chain smart contracts (Solidity Team, 2025). They facilitate the flow of essential data inputs to smart contracts, enabling them to execute predefined actions based on real-world events. However, when the validation process for oracle data is flawed, or when a contract overly relies on a single oracle, it introduces vulnerabilities into the contract's logic. For instance, if a contract fails to adequately validate data obtained from an oracle, it may accept inaccurate or malicious inputs, leading to incorrect contract execution. Similarly, depending solely on one oracle introduces a single point of failure, making the contract vulnerable to manipulation. In both cases, the contract's logic becomes compromised, as it cannot ensure the integrity of the data it relies upon for decision-making. In Listing 3, the “updatePrice” function does not validate the “newPrice” received from the oracle. As a result, any erroneous or malicious data provided by the oracle will be accepted by the contract logic and stored without verification. This oversight can result in unexpected logic outputs or other unintended behaviors, particularly if the contract relies on the price data for critical decision-making processes.

Listing 3: Incorrect Validation of Oracle Data

```
interface Oracle {
    function getPrice() external view returns
        (uint256);
}
contract PriceConsumer {
    address public oracle;
    uint256 public price;

    event PriceUpdated(uint256 newPrice);
    constructor(address _oracle) {
        oracle = _oracle;
    }

    function updatePrice() public {
        // Fetch price from the oracle
        uint256 newPrice =
            Oracle(oracle).getPrice();
        // No validation on the price data received
        // from the oracle
        price = newPrice;
        emit PriceUpdated(newPrice);
    }
    function getPrice() public view returns
        (uint256) {
        return price;
    }
}
```

5.1.5. Stale oracle data

This vulnerability arises when a smart contract relies on outdated data from oracles, leading to logical decisions based on obsolete information. For instance, trading contracts dependent on price feed oracles may execute transactions using outdated prices, resulting in unwanted logic. Moreover, inadequate handling of unexpected or malformed responses from oracles can cause unintended logic. For example, if an oracle returns a negative or zero value unexpectedly, it may disrupt the contract's behavior. Therefore, a contract logic should gracefully handle such scenarios is crucial for maintaining logic integrity and preventing undesirable outcomes.

5.1.6. Enumerating storage slot logic

The logic of enumerating slots in the contract storage is significant. Developers can enumerate their contract storage slots using a pattern that is well-known and predictable. Therefore, attackers can access or guess the storage content and manipulate it in their favor. This manipulation can disrupt the intended flow of the contract logic by altering critical data stored in specific storage slots. For instance, using common offsets for storing data crucial for the contract logic enables attackers to target specific storage slots and thus execute the logic as they wish.

5.1.7. Insecure upgrade authorization logic

This vulnerability concerns the logic managing the authorization of contract upgrades, allowing unauthorized parties to approve or trigger upgrades. It may arise from weak or predictable authorization mechanisms, which easily bypassed by users or unauthorized entities. Consequently, this can lead to unauthorized modifications, manipulation of contract logic, or even a complete contract takeover by malicious actors.

5.1.8. Inconsistent state transition checks

This logic-related vulnerability can occur during contract upgrades. It involves inadequate checks for the contract's current state or version, which can lead to unexpected behaviors in the execution of the contract logic. For instance, if the contract fails to properly verify its current state or version before initiating an upgrade, it may result in inconsistencies or vulnerabilities in the contract's logic. As a consequence, the contract may experience malfunctions, data corruption, or unexpected behaviors due to incorrect state transition checks during the upgrade process.

5.1.9. Insecure rollback mechanisms

This vulnerability occurs during the contract upgrading process, particularly when the implemented rollback mechanisms are inadequate or insecure. This can result in vulnerabilities or unexpected logical execution if upgrade failures or rollbacks occur. Rollback mechanisms in smart contracts typically refer to the ability to revert changes made to the contract state in case of errors or failures. For example, an incorrect implementation of robust rollback mechanisms may fail to properly revert state changes and ensure contract logic integrity in the event of upgrade failures or rollbacks, potentially leaving the contract in an inconsistent state. The impact of this vulnerability can manifest as contract malfunction due to inadequate rollback mechanisms that fail to properly handle upgrade failures or rollbacks.

Summary

- Code changes provide valuable insights into logic vulnerabilities, as evident from detecting numerous logic-related vulnerabilities. However, given the intricate nature of these vulnerabilities, it is essential to analyze code changes alongside the contract's source code and related contracts to gain a comprehensive understanding.
- Manual analysis of code changes, particularly when examining the business logic, proved challenging. Nonetheless, it revealed novel logic vulnerabilities, highlighting the depth of vulnerabilities that automated methods may overlook.
- Logic vulnerabilities in smart contracts range from syntactically detectable vulnerabilities, which are easily identified by tools or using regular expressions, to semantic-specific vulnerabilities that demand a profound understanding of the semantics of the contract and its business logic and context.

5.2. Answers to (RQ2): Automated detection of logic-related vulnerabilities

To answer RQ2, we utilized labeled vulnerabilities as described in Section 3 of the method. We split the data into a 75/25 ratio for training and evaluation, following the default setting of the `train_test_split` function from `scikitlearn`. The training method followed is outlined in Section 3. During the training process, we evaluated our approach and related models on an unseen test set to assess their accuracy in detecting various types of logic and general vulnerabilities within the contract code.

Table 6

Sóley performance metrics for different vulnerability categories.

	Precision	Recall	F1
Reentrancy (RE)	0.89	0.88	0.89
Uninitialized Local Variables (UL)	0.93	0.93	0.93
Recursive Calls In Loops(CLP)	0.92	0.95	0.94
Incorrect Low-Level Calls (LLC)	0.94	0.93	0.94
Locked Ether (LE)	0.97	0.91	0.94
Incorrect Equality Check (IE)	0.81	0.88	0.84

To effectively detect logic vulnerabilities within smart contracts, LLMs require a substantial number of labeled instances comprising numerous labeled instances of such vulnerabilities. Instead of a binary classification approach (vulnerable or not), we need multi-class classification to pinpoint the specific type of vulnerability present, if any. We have curated a selection of vulnerabilities, all of which include inline assembly fragments and are categorized as logic-related by Zhang et al. (2020), which demand nuanced comprehension of context and semantics. These include Locked Ether (LE) and Incorrect Equality Check (IE). In contrast, we have included three vulnerabilities with inline assembly fragments (i.e., Uninitialized Local Variables (UL), Recursive Calls in Loops (CLP), and Incorrect Low-Level Calls (LLC)). In addition, we have included Reentrancy (RE) vulnerability with inline assembly, as a recent empirical analysis by Chaliasos et al. (2024) highlighted that all preventable attacks on smart contracts were related to RE vulnerabilities. Moreover, we have omitted business logic-related vulnerabilities due to labeling challenges and the unique nature of each contract's logic. Our primary objective is to detect well-known logic vulnerabilities with inline assembly fragments that depend on contextual understanding and compare the performance of LLMs in automatically detecting them.

The results in Table 6 show that Sóley excels in detecting Uninitialized Local Variables (UL), Recursive Calls in Loops (CLP), and Incorrect Low-Level Calls (LLC). Precision and recall values consistently above 0.90, resulting in an F1-measure of 0.93 and above. This reveals a balanced performance, effectively identifying true positives while minimizing false positives and negatives. These vulnerabilities are characterized by recognizable patterns or behaviors. However, vulnerabilities such as LE and IE show slightly lower performance measures, indicating Sóley's challenge in accurately identifying instances of these vulnerabilities. For instance, LE vulnerabilities involve nuanced contextual considerations regarding fund locking and withdrawal conditions within smart contracts. Similarly, detecting IE vulnerabilities requires a deeper understanding of how the contract handles money or value comparisons. For RE vulnerabilities, although the precision and recall are slightly lower, Sóley still achieves a high F1-measure of 0.89. We believe this is due to potential variation in the entry points based on the contract logic. Nonetheless, Sóley demonstrates strong overall effectiveness in detecting vulnerabilities within smart contracts.

In Fig. 4, we present the confusion matrix of Sóley's classification results. We analyze the misclassification (i.e., false positives) to determine which vulnerabilities are being misclassified by Sóley. Notably, Sóley misclassifies IE as RE in 7% of cases. This suggests that many instances labeled as IE are instead identified as RE. The resulting misclassification between these two vulnerabilities can be due to several reasons. Firstly, IE and RE vulnerabilities might share similar patterns, making it difficult for the model to distinguish between them. For example, both vulnerabilities can involve complex control or data flow, which may confuse the model. Secondly, the training data might not adequately represent the differences between these vulnerabilities, causing the model to generalize poorly to new instances. Lastly, the simplicity of Sóley model architecture may prevent it from capturing nuanced differences between these two vulnerability types. However, since the

Output Class	RE	91%	2%	1%	3%	2%	1%
	UL	0%	90%	0%	7%	1%	0%
	CLP	2%	1%	94%	0%	1%	1%
	LLC	2%	0%	1%	96%	0%	1%
	LE	2%	3%	0%	0%	94%	1%
	IE	7%	0%	1%	3%	0%	89%
	Target Class	RE	UL	CLP	LLC	LE	IE

Fig. 4. Confusion matrix for the classification results of Sóley. Note: RE refers to Reentrancy, UL to Uninitialized Local Variables, CLP to Recursive Calls in Loops, LLC to Incorrect Low-Level Calls, LE to Locked Ether, and IE to Incorrect Equality Check.

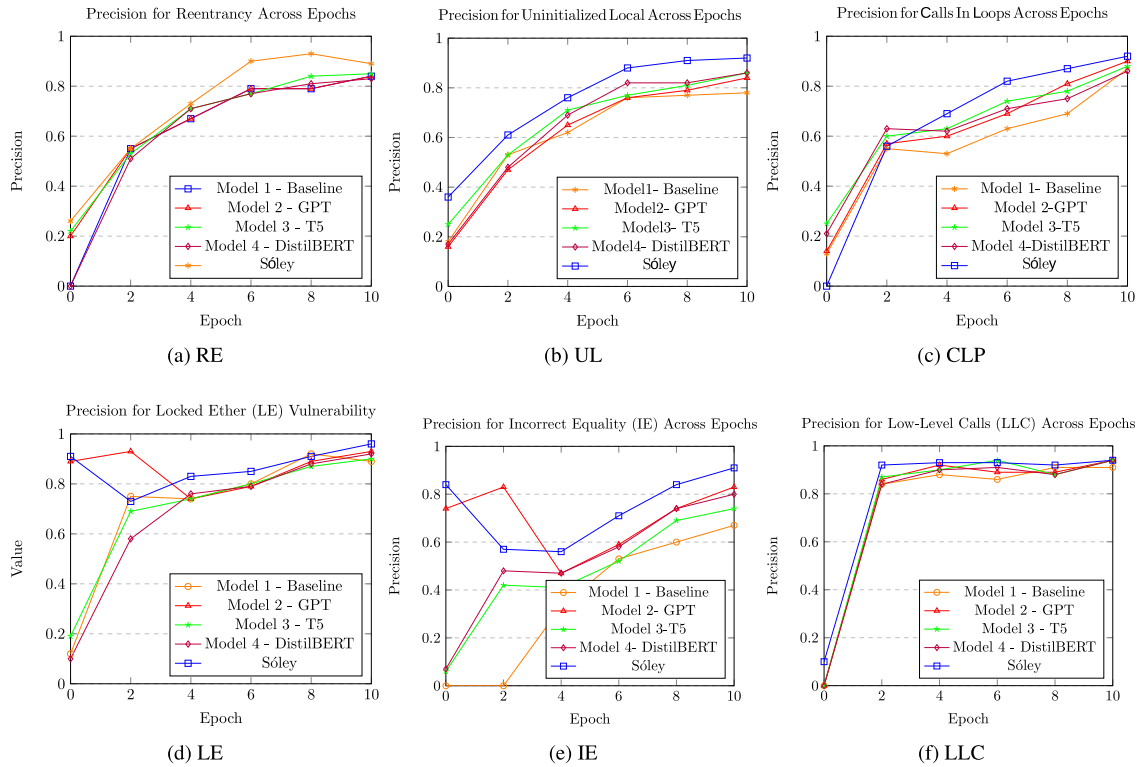


Fig. 5. Precision rates across epochs for selected vulnerabilities.

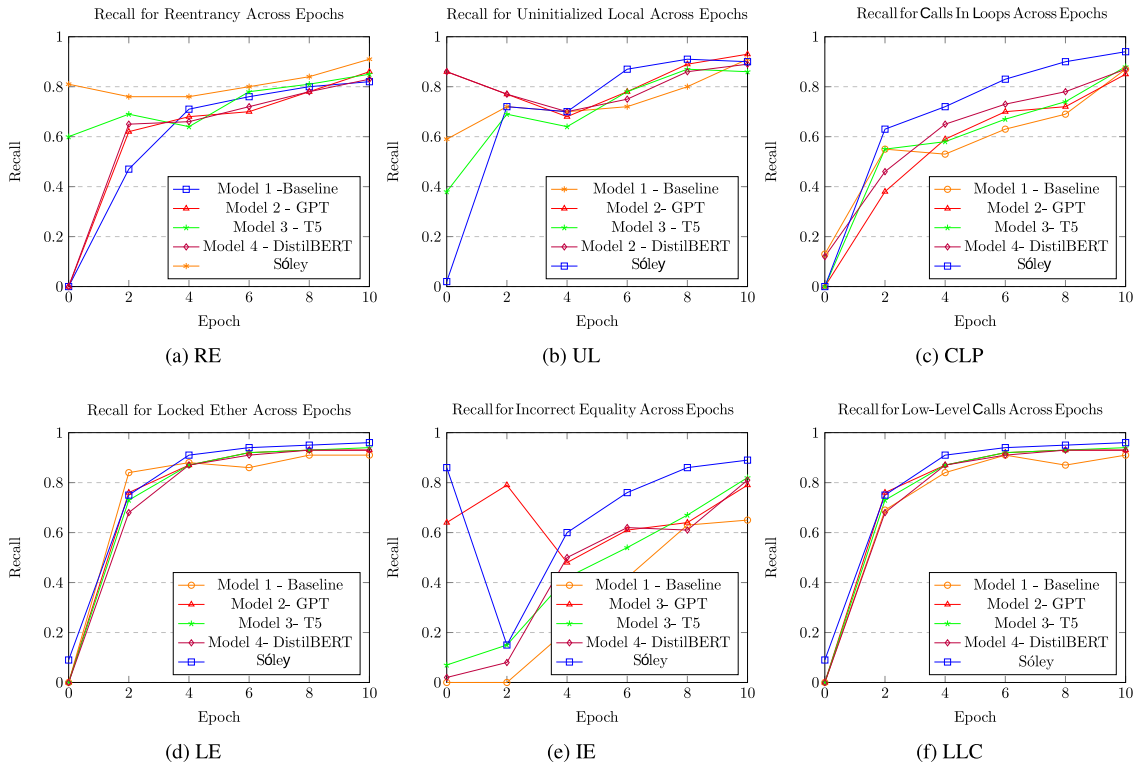


Fig. 6. Recall rates across epochs for selected vulnerabilities.

Table 7

Model evaluation in terms of F1-measure and accuracy (Acc.). Note: DBERT refers to DistilBERT. RE stands for Reentrancy, UL for Uninitialized Local Variables, CLP for Recursive Calls in Loops, LLC for Incorrect Low-Level Calls, LE for Locked Ether, and IE for IE Check.

	Epoch 10						
	RE	UL	CLP	LLC	LE	IE	Acc.
Baseline	0.78	0.84	0.87	0.91	0.89	0.66	0.84
GPT2	0.85	0.88	0.88	0.93	0.92	0.81	0.88
T5-Base	0.85	0.86	0.88	0.94	0.89	0.78	0.87
Sóley	0.9	0.91	0.93	0.95	0.95	0.9	0.93
DBERT	0.83	0.88	0.86	0.93	0.9	0.8	0.87

percentage of misclassification is relatively small, we believe that there are patterns in these two vulnerabilities that share similarities in terms of data flow and the training data does not provide the model with clearly distinct features.

Additionally, we observe a 3% misclassification rate from low-level calls to uninitialized local variables. The misclassification between these two types can be explained by instances where low-level function calls may share similarities with scenarios where variables are used without being properly initialized, such as both involving memory address access, leading to confusion.

To determine how well Sóley performs in comparison to the baseline and other related models, we fine-tuned the selected models for 10 epochs to maintain fair comparison. We also used the same hyperparameters for all models and the same training evaluation percentages. Table 7 compares the selected models in terms of F1-measure and accuracy at epoch number 10. Our results indicate that Sóley outperforms the selected models and baseline in terms of accuracy, with a

percentage of 5%–9% of improvement. We believe that is because our selected model was pre-trained on different programming languages, including object-oriented languages, that share similarities to Solidity syntax (Atzei et al., 2017a).

Summary

Sóley shows slight underperformance in distinguishing between Integer Equality (IE) and Reentrancy (RE) vulnerabilities. These limitations arise from the nuanced contextual considerations required for fund locking and withdrawal conditions in Locked Ether (LE), as well as the complexities involved in assessing value comparisons in IE cases. The primary reasons contributing to the limitations in distinguishing between IE and RE are as follows:

- **Complex Control Flow:** Complicated flows within the contract make detection challenging.
- **Shared Data Patterns:** Similar data flows can lead to misclassification.
- **Contextual Overlap:** There are overlapping contexts in fund handling and state changes.

Regarding the F1-measure, we observe that all models struggle with the RE, it is a result of determining the entry point in the logic of the contracts. So mostly the model identifies functions with the contract as susceptible to RE when they are not. Most of the models also face false positives in the IE, and that is because of erroneous identification of valid equality comparisons as vulnerabilities. The low-level calls vulnerability has the highest F1 measures across all models. This vulnerability arises when low-level function calls are made without proper validation, leading to security risks such as buffer overflows (Soud

Table 8
Model performance across vulnerabilities and epochs.

Vulnerability	Epoch	Metric	Baseline	GPT2	T5	Sóley	DBERT
Reentrancy	0	Precision	0.00	0.20	0.22	0.26	0.00
		Recall	0.00	0.00	0.60	0.81	0.00
		F1-measure	0.00	0.01	0.32	0.39	0.00
	6	Precision	0.76	0.79	0.77	0.90	0.77
		Recall	0.72	0.70	0.78	0.80	0.72
		F1-measure	0.74	0.74	0.78	0.85	0.74
	10	Precision	0.82	0.84	0.85	0.89	0.83
		Recall	0.74	0.86	0.85	0.91	0.83
		F1-measure	0.78	0.85	0.85	0.90	0.83
Uninitialized local variables	0	Precision	0.18	0.16	0.25	0.36	0.17
		Recall	0.59	0.86	0.38	0.02	0.86
		F1-measure	0.27	0.28	0.30	0.03	0.29
	6	Precision	0.76	0.76	0.75	0.88	0.82
		Recall	0.72	0.78	0.80	0.87	0.75
		F1-measure	0.74	0.77	0.77	0.88	0.78
	10	Precision	0.78	0.84	0.86	0.92	0.86
		Recall	0.91	0.93	0.86	0.90	0.89
		F1-measure	0.84	0.88	0.86	0.91	0.88
Recursive calls in loops	0	Precision	0.13	0.14	0.25	0.00	0.21
		Recall	0.23	0.00	0.00	0.00	0.12
		F1-measure	0.16	0.01	0.01	0.00	0.15
	6	Precision	0.63	0.69	0.74	0.82	0.71
		Recall	0.73	0.70	0.67	0.83	0.73
		F1-measure	0.68	0.69	0.70	0.83	0.72
	10	Precision	0.87	0.90	0.88	0.92	0.86
		Recall	0.86	0.85	0.88	0.94	0.87
		F1-measure	0.87	0.88	0.88	0.93	0.86
Incorrect low-level calls	0	Precision	0.00	0.00	0.00	0.10	0.00
		Recall	0.00	0.00	0.00	0.09	0.00
		F1-measure	0.00	0.00	0.00	0.09	0.00
	6	Precision	0.86	0.89	0.94	0.93	0.91
		Recall	0.91	0.92	0.92	0.94	0.91
		F1-measure	0.88	0.91	0.93	0.93	0.91
	10	Precision	0.91	0.94	0.94	0.94	0.94
		Recall	0.91	0.93	0.94	0.96	0.93
		F1-measure	0.91	0.93	0.94	0.95	0.93
Locked ether	0	Precision	0.12	0.13	0.19	0.34	0.10
		Recall	0.07	0.04	0.11	0.48	0.01
		F1-measure	0.09	0.06	0.14	0.40	0.02
	6	Precision	0.80	0.79	0.80	0.85	0.79
		Recall	0.81	0.83	0.83	0.90	0.86
		F1-measure	0.80	0.81	0.81	0.87	0.82
	10	Precision	0.89	0.93	0.90	0.96	0.92
		Recall	0.88	0.91	0.87	0.94	0.88
		F1-measure	0.89	0.92	0.89	0.95	0.90

et al., 2024) or injection attacks (Ghaleb and Pattabiraman, 2020). It seems that all models recognize LLC vulnerabilities, which are the easiest to detect. Finally, we observe that all models share a close trend regarding the detection performance for all vulnerabilities at epoch 10 in terms of F1-measures.

To study the performance of the selected models across different epochs we tracked the precision for all models per vulnerability, as shown in Fig. 5. The first six charts provide insights into the precision of various models across epochs for detecting different vulnerabilities. Across all vulnerabilities, we observe an overall increasing trend in precision as the training epochs progress. Therefore, while there may be variability in performance across different vulnerabilities, the models consistently improve in their ability to detect vulnerabilities over time, with some vulnerabilities being easier to detect than others. Similarly, recall rates in Fig. 6 reveal a notable increase in the performance of the selected models. However, for the IE vulnerability, we see an unusual

pattern where Sóley's recall starts high at 0.86, drops significantly at epoch 2, and then gradually recovers, indicating potential instability or sensitivity to early training stages. In contrast, DistilBERT exhibits a much lower starting recall for IE but demonstrates a steady improvement, highlighting its robustness in learning over time despite initial setbacks. T5 also shows a unique pattern with a slow start in recall for incorrect-equality, but catches up significantly by epoch 10. However, with our dataset, the models began to overfit after 10 epochs, leading us to stop at this point. We believe there is potential for further accuracy improvements with extended training and larger datasets, especially for large models such as T5 and GPT. Detailed metrics, including precision, recall, and F1-score across each epoch for all models, are presented in Tables 8 and 9, providing a comprehensive view of the improvements and performance variability of each model across the selected vulnerabilities at different epochs.

Table 9
Model performance across epochs for incorrect equality vulnerability.

Vulnerability	Epoch	Metric	Baseline	GPT2	T5	Sóley	DBERT
Incorrect equality check	0	Precision	0.00	0.18	0.06	0.03	0.07
		Recall	0.00	0.07	0.07	0.01	0.02
		F1-measure	0.00	0.10	0.06	0.01	0.03
	6	Precision	0.53	0.59	0.52	0.71	0.58
		Recall	0.42	0.61	0.54	0.76	0.62
		F1-measure	0.47	0.60	0.53	0.73	0.60
	10	Precision	0.67	0.83	0.74	0.91	0.80
		Recall	0.65	0.79	0.82	0.89	0.81
		F1-measure	0.66	0.81	0.78	0.90	0.80

Summary

- Our automated detection method for logic-related vulnerabilities, called Sóley, outperforms the baseline and the state-of-the-art related LLMs in terms of accuracy, with a percentage of 5%–9% of improvement.
- Our experiments demonstrate that LLMs can effectively detect several types of vulnerabilities with minimal feature engineering. However, all models face challenges in detecting Reentrancy (RE) vulnerabilities, mainly due to three reasons: insufficient context or variations in entry points based on the contract logic, or a lack of variety in the examples present in the training dataset.

5.3. Answers to (RQ3): Mitigation strategies in smart contracts

To answer this RQ, we comprehensively summarize common strategies followed by developers to address potential logic vulnerabilities based on the collected code changes available in our dataset. In the following, we provide detailed descriptions of each mitigation strategy. Moreover, many examples can be found in our dataset with the labeled code changes. Finally, a brief overview of the mitigation strategies is given in Table 10.

S1: Optimizing and Simplifying Bitwise Operations in Inline Assembly. This strategy involves reviewing and replacing unnecessary bitwise shift operations within inline assembly with built-in Solidity operators, such as `»` for bitwise right shifts. Developers simplify bitwise manipulation techniques in inline assembly and remove complex ones to reduce the potential for logic vulnerabilities arising from misunderstandings in the assembly code.

S2: Adding Checks for Low-Level Calls in Inline Assembly. Developers utilize additional checks and assertions when making low-level calls in inline assembly, as these calls can bypass the logic in Solidity and introduce vulnerabilities via unauthorized or unintended behavior.

S3: Removing Unnecessary Inline Assembly. When unnecessary inline assembly is used in the contract, we have noticed that developers either delete it or minimize the logic in the inline assembly.

S4: Invariant Validations in Upgradable Contracts. When performing contract upgrades, essential contract properties (i.e., invariants) must be preserved. These invariants include properties related to the contract such as interactions with external contracts, the contract state, and so on. Any inconsistency in these properties may affect the dependent logic and significantly impact the whole contract, resulting in vulnerabilities and unwanted behavior. This strategy includes using assert statements to ensure that these invariants are the same across all versions and that the functionality is consistent and unchanged.

S5: Adding Validation to Ensure Withdrawals. This strategy involves developers adding validation checks to withdrawal functions within the contract logic. These checks verify if the withdrawal amount exceeds the balance of the recipient. If the withdrawal amount exceeds

the balance, the transaction is reverted, and an appropriate error message is provided. Moreover, these checks are added to ensure that only authorized users can withdraw funds and that the withdrawal process is executed securely.

S6: Adding Checks for Resetting Token Allowances. This strategy is employed by developers to validate token allowance management functions to prevent unintended changes to token allowances, reducing the risk of potential logic vulnerabilities such as unauthorized token transfers. By adding specific assertions, developers ensure that resetting token allowances aligns with the contract's intended functionality.

S7: Adding Checks for Incorrect Fee Calculations. Developers modify the code to ensure accurate and reliable fee calculations by adding validation checks to fee calculation functions. This also includes adjusting the length and prefix calculations to ensure they are based on correct input values.

S8: Adding Checks for Handling Zero Balances.

Developers add checks to functions that handle token balances to ensure that zero balances are handled appropriately. These checks are added to manage cases where the balance of the recipient or the sender (`msg.sender`) is zero. Depending on the business logic of the contract, developers handle these cases differently, reducing the risk of potential vulnerabilities such as balance manipulation.

S9: Optimizing and Refactoring Interest Rate Calculation. Refactoring the interest rate calculation involves avoiding unnecessary intermediate steps and redundant operations. For instance, combining the calculation of interest rates for borrowing and lending into a single calculation while also adding proper checks for current interest rates and calculations.

S10: Ensuring Consistency Between Function Modifiers and State Mutability Specifiers. Addressing these types of logic vulnerabilities involves ensuring consistency between function modifiers and state mutability specifiers. This includes selecting the appropriate state mutability specifier (pure, view, payable, or nonpayable) based on the function's behavior and ensuring that the function modifier accurately reflects the function's interaction with external contracts or accounts.

S11: Validating Selection of Operations Based on Opcode Values. When a developer decides to use operations in their logic using opcodes, they validate the selection of operations to ensure that opcode values fall within the expected range. This ensures that only authorized operations are executed, reducing the risk of potential vulnerabilities.

S12: Accurate Parameter Handling in Function Invocations. We have noticed that some functionalities within the logic of the contract require certain inputs, either from inside or externally. Developers add validations for the parameters of such functions to ensure they adhere to expected formats, ranges, and constraints. Invalid or malicious inputs can lead to unexpected logic or vulnerabilities.

S14: Minimizing and Validation of External Function Calls. External calls are checked by developers in two steps: first, by checking the return value to verify whether the operation succeeded or failed, and second, by using `require` statements or conditional checks to validate return values. For example, if calling an external contract to

Table 10
Summary table of mitigation strategies for logic-related vulnerabilities.

Mitigation strategy	Description	Vulnerabilities mitigated
S1: Optimizing and Simplifying bitwise operations in inline assembly	Simplifies bitwise operations by using built-in Solidity operators instead of complex inline assembly.	Misunderstandings in assembly code leading to unintended behavior.
S2: Adding checks for low-level calls in inline assembly	Implements checks and assertions for low-level calls to prevent unauthorized behavior.	Logic bypass leading to unintended function execution.
S3: Removing unnecessary inline assembly	Deletes or minimizes unnecessary inline assembly to reduce complexity.	Increased risk of logic vulnerabilities due to complexity.
S4: Invariant validations in upgradable contracts	Ensures essential contract properties remain consistent across versions using assert statements.	Inconsistencies during contract upgrades affecting functionality.
S5: Adding validation to ensure withdrawals	Adds checks to withdrawal functions to verify balances and authorized users.	Unauthorized withdrawals or funds mismanagement.
S6: Adding checks for resetting token allowances	Validates token allowance management to prevent unintended changes.	Unauthorized token transfers due to logic flaws.
S7: Adding checks for incorrect fee calculations	Ensures accuracy in fee calculation functions through validation checks.	Miscalculations leading to financial discrepancies.
S8: Adding checks for handling zero balances	Implements checks to manage token balance operations appropriately.	Vulnerabilities related to zero balance manipulation.
S9: Optimizing and Refactoring interest rate calculation	Refactors interest rate calculations to streamline the process and ensure accuracy.	Errors in interest rate logic leading to financial losses.
S10: Ensuring consistency between function modifiers and state mutability specifiers	Aligns function modifiers with state mutability specifiers to reflect the function's behavior accurately.	Logic inconsistencies and potential unauthorized access.
S11: Validating selection of operations based on opcode values	Validates opcode selection to ensure operations fall within expected ranges.	Execution of unauthorized or unexpected operations.
S12: Accurate parameter handling in function invocations	Adds parameter validation for function inputs to prevent invalid or malicious entries.	Unexpected behavior from improper function inputs.
S14: Minimizing and Validation of external function calls	Checks return values of external calls and validates them with require statements.	Failures in external interactions leading to inconsistent state.
S15: Removing or limiting complex data structures	Reduces complexity of data structures to minimize unintended behaviors.	Increased chance of errors due to complex interactions.

transfer tokens, a developer checks if the transfer was successful before proceeding with further operations.

S15: Removing or Limiting Complex Data Structures. Complex data structures, such as nested arrays, increase the likelihood of unintended behaviors in the contract logic. Developers remove or limit the complexity of these data structures. This mitigation strategy often entails refactoring the code to replace complex data structures with simpler alternatives, such as using flat arrays instead of nested structures. Developers also impose constraints on the depth or complexity of data structures to prevent them from becoming overly intricate.

6. Discussions and implications

This section discusses the limitations and implications of our findings.

Our method has several limitations, primarily centered around the reliance on labeling through regular expressions, state-of-the-art tools, and manual labeling. Our approach includes multiple steps to identify vulnerabilities in both smart contracts and code changes. Initially, we utilize regular expressions, but this method often results in high false positives. To mitigate this, we focus on vulnerabilities that are also identified by state-of-the-art tools. However, not all vulnerabilities, especially those related to logic, can be effectively identified using regular expressions alone. Variants of vulnerability types that involve complex logic may remain undetected by this method. Furthermore, while state-of-the-art tools are robust, they may not encompass every type of vulnerability, particularly those originating from logical errors. Hence, our method includes labeling vulnerabilities based on the intersection of findings from both regular expressions and state-of-the-art tools.

In essence, there may exist vulnerabilities that neither method detects or new types of vulnerabilities that were previously unknown. Our manual checks have identified and defined new vulnerabilities. However, manually labeling a dataset as extensive as ours poses significant challenges, compounded by the absence of an automated method for double-checking identified vulnerabilities. Therefore, adopting a combined approach that integrates tools and regular expressions is necessary to effectively identify known vulnerabilities. To minimize this problem during testing and training the model, we have focused primarily on well-known logic vulnerabilities such as locked ether and incorrect equality sanity checks.

The second limitation is capturing the context of vulnerabilities. The diff and selected lines describing vulnerabilities and their context in our work may have some limitations. There are instances where vulnerabilities depend on interactions across extensive portions of code, spanning large contracts or multiple libraries included in those contracts. Our model may not fully understand the implications of such widespread dependencies, as it was not trained on these specific scenarios.

Furthermore, while we considered various types of vulnerabilities, the context in which they occur may not always be fully represented. Therefore, we focus on typical and crucial logic-related vulnerabilities defined in the literature for which we have confidence in our model's training on a substantial dataset. Moreover, our dataset is restricted to mitigations that developers have implemented in their code changes (diffs). This means that any unrecognized mitigations or ignored vulnerabilities remain unseen, creating gaps in our understanding of new vulnerabilities or mitigations. Therefore, we deliberately focused only on the mitigations and vulnerabilities that developers have fixed, similar to the approach used by [Liu et al. \(2018\)](#). This method helps us avoid false positives by considering only the issues that developers

have addressed, but it also means we might miss vulnerabilities that developers did not recognize, fix, or mitigate.

Finally, our model exhibits faster training times due to its architecture being a simplified version of the BERT architecture. However, it is important to note that its smaller size could make it more susceptible to overfitting compared to larger models. We anticipate that a larger dataset and more extensive training could enable larger models to achieve similar levels of accuracy.

Summary of Limitations

- Although Sóley is effective in detecting logic-related vulnerabilities, it struggles to identify certain classes of logic vulnerabilities, such as Incorrect Equality (IE). This limitation stems from the complexities involved in accurately assessing value comparisons in IE cases.

6.1. Implications

For Researchers. Researchers can utilize Sóley to expand the scope of vulnerability detection in smart contracts. Utilizing minimal feature engineering techniques such as tokenization and slicing, our results show that Sóley and LLMs offer promising avenues for accurately detecting diverse vulnerabilities. We believe additional exploring and the integration of control flow or data flows with LLMs could enhance vulnerability detection by integrating more advanced feature engineering. Adding more context, such as comments alongside code snippets or additional lines, could improve the semantic understanding of LLMs and enhance their detection capabilities. To address the challenge of detecting complex business logic vulnerabilities, researchers might explore transfer learning or fine-tuning Sóley on domain-specific datasets, potentially drawn from real-world contracts with known business logic anomalies. Incorporating reinforcement learning through iterative feedback loops could also help Sóley adapt its detection approach based on prior successes and challenges. Additionally, attention mechanisms focused on critical code paths could make Sóley more responsive to high-risk, logic-dependent vulnerabilities, such as those found in Ether transactions or asset transfer functions, where complex business logic errors are more common. Together, these approaches could improve the model's ability to identify domain-specific vulnerabilities that often go undetected in syntactic analysis alone. Moreover, Sóley can be used to curate datasets and generate labeled datasets with more accurate and semantically rich vulnerabilities. Future research could focus on developing automated approaches that utilize code changes to identify vulnerabilities in smart contracts at an early stage. There is also a need for further exploration into automating the patching of smart contract vulnerabilities using code changes, as well as investigating the efficacy of different fix strategies commonly employed by developers. Lastly, implementing adaptive learning techniques from detected vulnerabilities may enable Sóley and other LLMs to adjust dynamically to emerging threats, resulting in more robust detection capabilities over time.

For Practitioners. Developing automated analysis tools that integrate Sóley with version control systems such as Git can improve detecting logic vulnerabilities in smart contracts. Additionally, tool builders can extend existing analysis tools by integrating Sóley to detect logic-related vulnerabilities through plugins or extensions. That is, Sóley can be used to detect logic vulnerabilities before a smart contract is deployed, in similar ways as static analysis tools or automated testing tools are widely used by developers. For instance, Sóley could be run as a GitHub Action¹⁰ or as a part of a general CI/CD pipeline on commits, pull requests or merge requests. To optimize resources, Sóley could also

be used selectively on high-risk or high-value contracts, which require more rigorous analysis. Alternatively, exploring lightweight model variants could provide cost-effective integrations without compromising detection capabilities.

While this integration can improve the vulnerability detection and mitigation process, practitioners may face a learning curve in interpreting LLM outputs effectively. This learning curve can stem from the inherent complexity of the model's responses, which often require a nuanced understanding of both the technology and the specific context of the smart contracts being analyzed. Practitioners should possess a solid foundation in smart contract development and security principles to accurately interpret these outputs and integrate them into their workflows. To this end, we are working on providing additional tools and documentation that make Sóley more usable by practitioners, e.g., by providing a more practitioner-friendly tool output.

Tool builders in the space of smart contract can augment their tools' capabilities to automate the patching of identified vulnerabilities. That is, by using the Sóley dataset and detection process, tools can provide developers with code suggestions for fixing their smart contracts, based on examples in our dataset from similar logic vulnerabilities. In addition to automatically suggesting code, the in-depth logic-related examples in our dataset can help this integration.

7. Related work

This section describes studies related to our work, including empirical studies on logic code vulnerabilities in smart contracts, LLMs in smart contract vulnerability detection, and datasets related to our research.

7.1. Empirical studies on logic vulnerabilities in smart contracts

In a study by Chaliasos et al. (2024), an empirical evaluation of existing tools for smart contract security was conducted. Their evaluation indicates that these tools often generate numerous insignificant reports, leading to an overwhelming number of false positives. Moreover, Chaliasos et al. highlight the inefficiency of these tools in detecting logic-related vulnerabilities, sanity checks, and logic errors. The study also shows that practitioners identify logic-related vulnerabilities and protocol layer vulnerabilities as significant threats that are not adequately addressed by existing security tools. Soud et al. (2024) defined logic vulnerabilities as inconsistencies with the contract and the programmer's intention. Among the identified logic-related vulnerabilities are Greedy Contract (i.e., locked ether), which arises when the contract logic only locks Ether; Transaction Order Dependency, occurring when a contract's logic depends on the order of transaction execution within a block; Call to the Unknown vulnerability, where a function unexpectedly invokes the recipient's fallback function, potentially introducing malicious code; and the DoS by External Contract vulnerability. Zhang et al. (2023) categorize 26 smart contract vulnerabilities into three groups. The first group includes hard-to-exploit or doubtful vulnerabilities. The second group is detectable by static analysis tools. The third group involves business logic vulnerabilities, such as price manipulation, which require high-level semantical oracles and are generally in the decision logic, branching, sequencing, or a computational algorithm, as found in natural language specifications or implementation language. The study identified four primary logic-related categories in smart contracts; including assembly code (i.e., inline assembly), DoS, fairness, and storage vulnerabilities.

¹⁰ <https://github.com/features/actions>.

Table 11
Comparative analysis of Sóley and existing static analysis tools for smart contract vulnerability detection.

Feature	Sóley	Mythril (Mueller, 2018)	Oyente (Luu et al., 2016)	SmartCheck (Tikhomirov et al., 2018a)	Slither (Feist et al., 2019)	Osiris (Torres et al., 2018)
Method used	LLM-based detection	Symbolic execution	Symbolic execution	Pattern matching	Static analysis	Fuzzing and static analysis
Solidity contract analysis	Operates on solidity code	Bytecode conversion	Bytecode conversion	Operates on solidity code	Operates on solidity code	Bytecode conversion
Inline assembly vulnerability detection	Detects logic-related vulnerabilities with inline assembly fragments	Limited; indirect through heuristics	Limited; indirect through heuristics	None	Limited	None
Generalizability	High	Low	Low	Low	Low	Low
Scalability	High	Low	Low	Moderate	Moderate	Moderate
Learning new patterns	High; adapts dynamically and identifies evolving patterns	Low	Low	Moderate; needs manual identification	Moderate; needs manual identification	Low

7.2. Detecting smart contract code vulnerabilities

Detecting smart contract vulnerabilities has attracted significant attention recently. Several tools have emerged to analyze smart contract code. These tools can generally be categorized into two main groups: traditional static and dynamic analysis tools and tools based on machine learning and large language models (LLMs).

Solidity vulnerability detection via traditional code analysis tools. Static analysis tools such as Slither (Feist et al., 2019) and smartcheck (Tikhomirov et al., 2018a) are designed to statistically analyze smart contracts, detecting security vulnerabilities and bad coding patterns. Oyente (Luu et al., 2016) is among the earliest and most widely used smart contract analysis tools. Oyente leverages symbolic execution to analyze EVM bytecode and to detect potential control flow vulnerabilities. Mythril (Mueller, 2018) is a symbolic execution tool that analyzes EVM bytecode by combining symbolic execution, taint analysis, and control flow analysis to detect various vulnerabilities in smart contracts, including overflow/underflow and the use of tx.origin. Solhint (Protofire, 2025) detects syntax-related vulnerabilities through static checks using a wide range of rules. Additionally, Osiris (Torres et al., 2018) facilitates integer vulnerability detection in Solidity by employing symbolic execution combined with taint analysis. Dynamic analysis tools such as Maian (Nikolić et al., 2018) categorize vulnerable smart contracts into three main types: suicidal contracts, prodigal contracts, and greedy contracts. Maian analyzes Solidity contracts through dynamic analysis on a private blockchain to reduce the number of false positives. However, despite the effectiveness of these tools, Chaliasos et al. (2024)’s empirical evaluation of these tools highlights that they do not adequately address logic-related vulnerabilities, which are often the root cause of high-impact attacks. Additionally, Zhang et al. (2023) noted that more than 80% of exploitable bugs remain undetectable by automated means. Moreover, the complex low-level instructions of inline assembly are only partially supported by existing static analysis tools (Chaliasos et al., 2024), and no existing tool in the available body of knowledge is designed to detect vulnerabilities with inline assembly fragments in smart contracts by the time of writing this paper.

Table 11 compares the key features of Sóley against selected static analysis tools for smart contract vulnerability detection, including Mythril, Oyente, SmartCheck, Slither, and Osiris. Sóley’s method is based on large language models (LLMs) with minimal feature engineering, enabling direct analysis of Solidity code alongside inline assembly fragments written in the Yul low-level language. In terms of Solidity-level analysis, SmartCheck and Slither also provide contract code analysis, focusing on static analysis and pattern matching. However, these tools lack the ability to analyze inline assembly fragments or the Yul language. As a result, they generally generate a warning whenever

an “assembly” code token is encountered in the contract (Chaliasos et al., 2024). Tools such as Mythril and Oyente rely on bytecode analysis, which requires an intermediate conversion from Solidity to bytecode. This process can result in losing semantics from the original code. In contrast, Sóley operates directly on high-level Solidity code, preserving the original semantics and flow of the contract while significantly reducing analysis time. Moreover, Sóley demonstrates high generalizability (Hu et al., 2023), adapting dynamically to evolving threats by learning new patterns and understanding the semantics and context of vulnerabilities. This makes it well-suited for handling previously unseen vulnerabilities, as the evaluation results show. In contrast, traditional static analysis tools are generally limited to pre-defined vulnerability patterns and rules. Static analysis tools such as Slither and SmartCheck have low adaptability due to their reliance on manually updated rule sets, which can address new vulnerabilities only with significant manual effort. Similarly, static analysis tools that focus on bytecode have low adaptability, as their techniques are harder to update for evolving threats. Sóley scales better than static analysis tools due to its lightweight, LLM-driven architecture, which allows it to handle diverse contracts efficiently (Li et al., 2024; Cui, 2024). In contrast, static analysis tools are constrained by the computational overhead of techniques such as symbolic execution or fuzzing. Finally, Sóley has the potential for higher interpretability (Hu et al., 2023), as it is based on LLMs, which could allow it to generate detailed explanations for vulnerabilities, reason about new vulnerability patterns, and suggest repair code segments. However, static analysis tools lack built-in support for interpretability and require external tools for deeper insights.

Solidity Vulnerability Detection using LLMs and transformer models.

Sun et al. (2024) introduced GPTScan, a tool that combines GPT with static analysis. GPTScan prompts GPT to automatically recognize scenarios related to logic vulnerabilities in smart contracts. Additionally, GPT is trained to identify key variables and statements, which are then verified through static confirmation. Hu et al. (2023) proposed GPTLENS based on GPT-4 as an auditor, analyzing smart contracts, and a critic, reviewing the audits. Their empirical findings indicate that GPTLENS brings significant enhancements compared to the traditional detection approaches. David et al. (2023) examined the feasibility of utilizing LLMs for smart contract security audits, evaluating them on 52 compromised Decentralized Finance (DeFi) smart contracts. Their findings indicate that GPT-4 and Claude models correctly identify vulnerabilities in 40% of cases but exhibit a notable false positive rate, necessitating manual auditor involvement. Sun et al. (2023) introduced ASSBERT framework, designed for smart contract

vulnerability classification based on active and semi-supervised bidirectional encoder representations from transformers (BERT) network. Jeon (Jeon et al., 2021) proposed SmartConDetect, a static analysis tool designed to detect security vulnerabilities in Solidity-based smart contracts. Utilizing a pre-trained BERT model, SmartConDetect extracts code fragments from smart contracts and identifies vulnerable code patterns. Kim et al. (2024) explored the use of fine-tuned transformer encoder models for detecting vulnerabilities in Ethereum smart contracts. Their evaluation across three datasets demonstrates that balancing data significantly enhances model performance, particularly in detecting minority class vulnerabilities. Their study highlights the importance of dataset quality and the effectiveness of transformer architectures in improving smart contract security. Recently, Liu et al. (2024) surveyed the body of knowledge related to the applications of Transformer models in blockchain technology. In their work, they cover a broad range of blockchain applications, including anomaly detection, smart contract security analysis, cryptocurrency prediction and trend analysis, and code summary generation. The paper specifically explores the application of various Transformer-based models, such as BERT, GPT, RoBERTa, and T5, to improve blockchain processes, particularly the analysis and execution of smart contracts. Their paper also discusses potential future research directions, emphasizing the need for a deeper exploration of Transformer architectures to better adapt them to specific blockchain and smart contract applications.

Existing approaches mainly detect Solidity vulnerabilities, yet few are designed specifically to detect vulnerabilities within the logic and inline assembly of the contract. Although the nature of logic-related vulnerabilities, sanity checks, and inline assembly vulnerabilities, making it more challenging to detect (Chaliasos et al., 2024, 2022).

7.3. Solidity vulnerability datasets

Several studies in the literature focused on datasets for Solidity vulnerabilities. Durieux et al. (2020) curated a dataset with 69 annotated vulnerable Solidity contracts obtained from Etherscan. Their dataset includes labels specifying the location and category of the vulnerabilities. Moreover, The authors provided an unlabeled dataset containing 47,518 contracts. Soud et al. (2023) constructed a dataset comprising approximately 6500 vulnerabilities and corresponding fix pairs. Chaliasos et al. (2022) constructed a dataset of Solidity smart contracts containing 12.4 million contracts. However, there are no available datasets specifically about logic vulnerabilities or logic-related inline assembly, nor a dataset with labeled vulnerabilities with code snippets of the vulnerable section.

8. Threats to validity

This section addresses challenges to the internal, construct, and external validity. To identify validity threats in our study, we utilized the standard methodology proposed by Feldt and Magazinius (2010).

Internal Validity: Labeling of vulnerabilities in large quantities poses a threat to internal validity due to the time and resource-intensive nature of the task. To mitigate the risk of incorrectly labeling contracts, we employed regular expressions and validated the results using two established tools, SmartCheck and Slither. Another internal validity concern relates to the availability of training data. Given the specificity of our task, obtaining a large amount of suitable data related to logic vulnerabilities with inline assembly was challenging. Addressing this, we selected vulnerabilities classified in the literature as logic vulnerabilities and compared them with other types of vulnerabilities, all with inline assembly fragments. The selection of models also presents a potential threat to internal validity. To mitigate this, we chose a range of well-reviewed LLMs. Additionally, there is a concern about the reliability of the quantitative analysis, considering the potential presence of unidentified vulnerabilities within the contracts. To address this, two experts reviewed the identified vulnerabilities from the manual labeling

process. Furthermore, we have made the raw data openly accessible, allowing other researchers and users to validate the results.

Construct Validity: This could be the choice of evaluation metrics in our study. Nevertheless, we minimized this threat by selecting widely recognized metrics such as precision, recall, and F1-measure. These metrics have been extensively employed in previous research, including the baseline. Another consideration regarding the construct validity of our study is the distribution of selected vulnerabilities, which may vary in quantity within the dataset. To ensure fair training and balanced data representation, we standardized the number of instances across all vulnerabilities, despite their varying occurrences in the dataset. The consistent high performance across all vulnerabilities suggests the efficacy of our approach. Additionally, to address concerns about training fairness across models, we maintained consistency by using identical hyperparameters, training datasets, optimizers, and the same machine for all models.

External Validity: A potential challenge to external validity is associated with the concern that the Ethereum smart contracts included in this study may not be a precise representation of all smart contracts across various blockchain platforms, such as Hyperledger Fabric or other contracts on different blockchains. Consequently, our quantitative analysis and approach may not accurately reflect the code changes or fixes in contracts beyond Ethereum smart contracts.

9. Conclusion

Securing smart contracts has attracted many researchers and practitioners. One important aspect of improving smart contract security is addressing logic-related code vulnerabilities, which often serve as the root cause of high-impact cyberattacks. In this work, we introduce Sóley, a novel approach for detecting logic-related vulnerabilities in Ethereum smart contracts. Sóley bridges the gap between traditional smart contract static analysis tools and the complex demands of logic vulnerability detection by leveraging LLMs with minimal feature engineering, allowing it to efficiently identify logic vulnerabilities where other static analysis tools may fall short. Our findings demonstrate that Sóley outperforms existing baselines and several LLMs in automatically detecting these vulnerabilities. We further investigate logic-related vulnerabilities based on code changes in smart contracts collected from GitHub. Additionally, we explore and identify mitigation strategies for these code vulnerabilities based on developers' code changes. Notably, without requiring extensive feature engineering, LLMs performed effectively in this task, as evident from the results. In the future, we plan to expand Sóley's capabilities to allow for the detection and patching of additional types of logic-related vulnerabilities to further advance Ethereum smart contract security. We also plan to expand our dataset by incorporating verified contracts from platforms like Etherscan, enabling broader validation of Sóley across the diverse, real-world Ethereum blockchain environment.

CRediT authorship contribution statement

Majd Soud: Writing – review & editing, Writing – original draft, Project administration, Methodology, Investigation, Data curation. **Walteri Nuutinen:** Validation, Software, Methodology, Investigation, Data curation. **Grischa Liebel:** Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: no competing interests. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

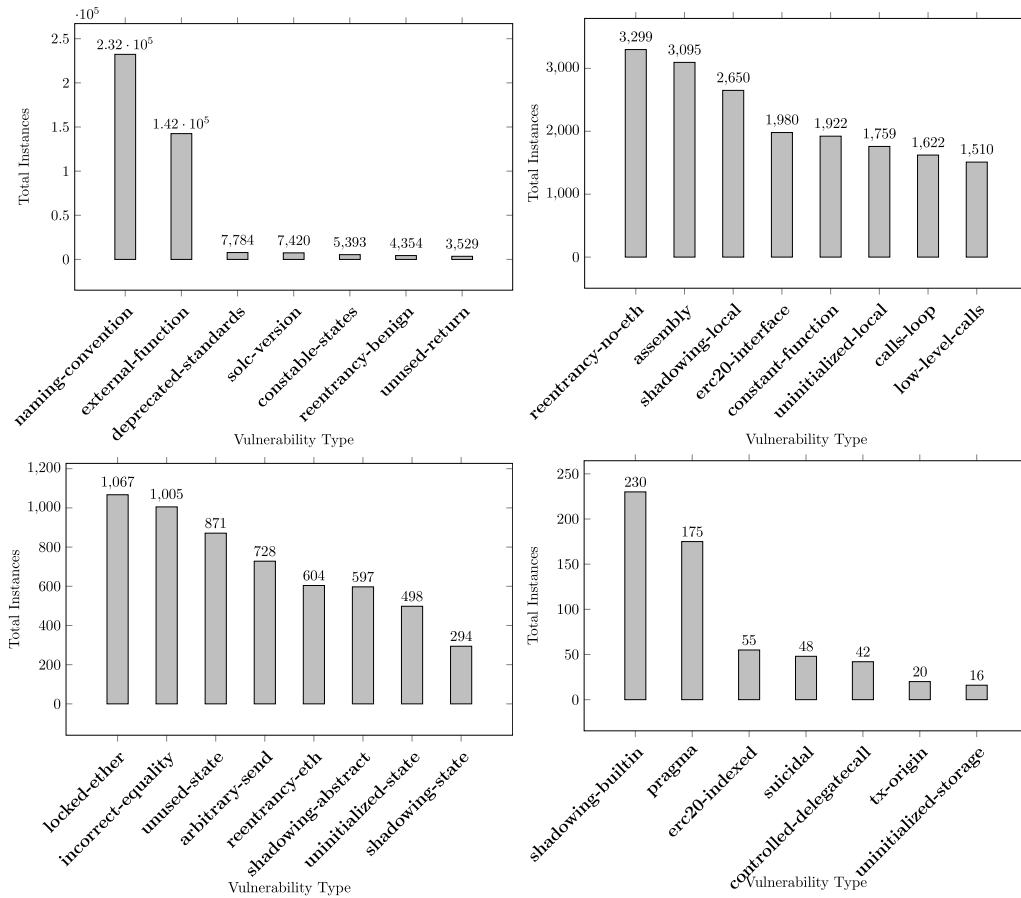


Fig. A.7. Vulnerability distribution in Sóley dataset.

References

- Antonopoulos, A.M., Wood, G., 2018. *Mastering Ethereum: Building Smart Contracts and Dapps*. O'Reilly Media.
- Atzei, N., Bartoletti, M., Cimoli, T., 2017a. A survey of attacks on ethereum smart contracts (sok). In: *Principles of Security and Trust: 6th International Conference, POST 2017, Held As Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April (2017) 22-29, Proceedings 6*. Springer, pp. 164–186.
- Atzei, N., Bartoletti, M., Cimoli, T., 2017b. A survey of attacks on ethereum smart contracts (sok). pp. 164–186. http://dx.doi.org/10.1007/978-3-662-54455-6_8.
- Brown, T.B., 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*.
- Buterin, V., et al., 2014. A Next-Generation Smart Contract and Decentralized Application Platform. White Paper 3, pp. 2–1.
- Chaliasos, S., Charalambous, M.A., Zhou, L., Galanopoulou, R., Gervais, A., Mitropoulos, D., Livshits, B., 2024. Smart contract and defi security tools: Do they meet the needs of practitioners? In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. pp. 1–13.
- Chaliasos, S., Gervais, A., Livshits, B., 2022. A study of inline assembly in solidity smart contracts. *Proc. ACM Program. Lang.* 6, 1123–1149.
- Chen, T., Feng, Y., Li, Z., Zhou, H., Luo, X., Li, X., Xiao, X., Chen, J., Zhang, X., 2020a. Gaschecker: Scalable analysis for discovering gas-inefficient smart contracts. *IEEE Trans. Emerg. Top. Comput.* 9, 1433–1448.
- Chen, H., Pendleton, M., Njilla, L., Xu, S., 2020b. A survey on ethereum systems security: Vulnerabilities, attacks, and defenses. *ACM Comput. Surv.* 53, 1–43.
- Clack, C.D., Bakshi, V.A., Braine, L., 2016. Smart contract templates: foundations, design landscape and research directions. *arXiv preprint arXiv:1608.00771*.
- Community, D., 2024. Erc-20, ERC-721, and other smart contract standards explained.
- Cui, H., 2024. Can large language model replace static analysis tools. In: *International Conference on Computer Network Security and Software Engineering (CNSSE 2024)*. Vol. 13175, SPIE, pp. 320–325.
- David, I., Zhou, L., Qin, K., Song, D., Cavallaro, L., Gervais, A., 2023. Do you still need a manual smart contract audit? *arXiv preprint arXiv:2306.12338*.
- Deconstructing the dao attack, 2016. Deconstructing the dao attack: A brief code tour. URL: <https://web.archive.org/web/20180128074919/http://vessenes.com/deconstructing-the-dao-attack-a-brief-code-tour/>.
- Destefanis, G., Marchesi, M., Ortu, M., Tonelli, R., Bracciali, A., Hierons, R., 2018. Smart contracts vulnerabilities: a call for blockchain software engineering? In: *2018 International Workshop on Blockchain Oriented Software Engineering. IWBOSE, IEEE*, pp. 19–25.
- Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P., 2020. Empirical review of automated analysis tools on 47 587 ethereum smart contracts. In: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. pp. 530–541.
- Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P., 2025. Smartbugs. , <https://github.com/smartbugs/smartbugs>.
- Ethereum, 2024. Ethereum GitHub repository. URL: <https://github.com/ethereum>. (Accessed June 2024).
- Ethereum Docs, 2023. Ethereum accounts. URL: <https://ethereum.org/en/developers/docs/accounts/>.
- Feist, J., Grieco, G., Groce, A., 2019. Slither: a static analysis framework for smart contracts. In: *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. pp. 8–15.
- Feldt, R., Magazinius, A., 2010. Validity threats in empirical software engineering research-an initial survey. In: *Seke*. pp. 374–379.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.
- Ghaleb, A., Pattabiraman, K., 2020. How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In: *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. pp. 415–427.

2016. Governmental contract. https://www.reddit.com/r/ethereum/comments/4ghzhv/governmentals_1100_eth_jackpot_payout_is_stuck.
- Goyal, P., 2017. Accurate, large minibatch sg d: training imagenet in 1 h. arXiv preprint arXiv:1706.02677.
- Hirai, Y., 2017. Defining the ethereum virtual machine for interactive theorem provers. In: Financial Cryptography and Data Security: FC 2017 International Workshops, WAHC, BITCOIN, VOTING, WTSC, and TA, Sliema, Malta, April 7 2017, Revised Selected Papers 21. Springer, pp. 520–535.
- Hu, S., Huang, T., İlhan, F., Tekin, S.F., Liu, L., 2023. Large language model-powered smart contract vulnerability detection: New perspectives. arXiv preprint arXiv:2310.01152.
- Hugging Face, 2024. CodeBERTa-small-v1. <https://huggingface.co/huggingface/CodeBERTa-small-v1>. (Accessed 21 May 2024), 2025.
- Jeon, S., Lee, G., Kim, H., Woo, S.S., 2021. Smartcondetect: Highly accurate smart contract code vulnerability detection mechanism using bert. KDD Work. Program. Lang. Process..
- Kim, J., Lee, S., Kim, H., et al., 2024. Robust vulnerability detection in solidity-based ethereum smart contracts using fine-tuned transformer encoder models. IEEE Access.
- Li, Z., Dutta, S., Naik, M., 2024. Llm-assisted static analysis for detecting security vulnerabilities. arXiv preprint arXiv:2405.17238.
- Liu, K., Kim, D., Bissyandé, T.F., Yoo, S., Le Traon, Y., 2018. Mining fix patterns for findbugs violations. IEEE Trans. Softw. Eng. 47, 165–188.
- Liu, T., Wang, Y., Sun, J., Tian, Y., Huang, Y., Xue, T., Li, P., Liu, Y., 2024. The role of transformer models in advancing blockchain technology: A systematic survey. arXiv preprint arXiv:2409.02139.
- Luu, L., Chu, D.-H., Olickel, H., Saxena, P., Hobor, A., 2016. Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 254–269.
- Metcalfe, W., et al., 2020. Ethereum, smart contracts, dapps. Blockchain Crypt Curr. 77, 77–93.
- Morrison, P., Herzig, K., Murphy, B., Williams, L., 2015. Challenges with applying vulnerability prediction models. In: Proceedings of the 2015 Symposium and Bootcamp on the Science of Security. pp. 1–9.
- Mueller, B., 2018. Smashing ethereum smart contracts for fun and real profit. HITB SECCONF Amst. 9, 4–17.
- Nakamoto, S., 2008. Bitcoin: A peer-to-peer electronic cash system. Decentralized Bus. Rev..
- Nikolić, I., Kolluri, A., Sergey, I., Saxena, P., Hobor, A., 2018. Finding the greedy, prodigal, and suicidal contracts at scale. In: Proceedings of the 34th Annual Computer Security Applications Conference. pp. 653–663.
- OpenAI, 2023. OpenAI Gpt-4: Technical report. URL: <https://openai.com/research/gpt-4>. (Accessed 07 June 2024).
- Perez, A.J., Zeadally, S., 2022. Secure and privacy-preserving crowdsensing using smart contracts: Issues and solutions. Comput. Sci. Rev. 43, 100450.
- Protofire, 2025. Solhint. <https://github.com/protofire/solhint>.
- Radford, A., Narasimhan, K., 2018. Improving language understanding by generative pre-training. URL: <https://api.semanticscholar.org/CorpusID:49313245>.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. J. Mach. Learn. Res. 21, 5485–5551.
- Rogers, A., Kovaleva, O., Rumshisky, A., 2020. A primer in bertology: What we know about how bert works. arXiv:2002.12327.
- Sanh, V., Debut, L., Chaumond, J., Wolf, T., 2019. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.
- Service, R.W., 2009. Book review: Corbin, j., & strauss, a. (2008) basics of qualitative research: Techniques and procedures for developing grounded theory, thousand oaks, ca: Sage. Organ. Res. Methods 12, 614–617.
- SolAI, 2024. Sóley: Identification and automated detection of logic vulnerabilities in ethereum smart contracts via large language models. URL: <https://figshare.com/s/efa4967b9eeb9dc6ee96>. (accessed 13 June 2024).
- Solidity Team, 2025. Solidity compiler documentation. <https://docs.soliditylang.org>.
- Soud, M., Liebel, G., Hamdaqa, M., 2024. A fly in the ointment: an empirical study on the characteristics of ethereum smart contract code weaknesses. Empir. Softw. Eng. 29, 13.
- Soud, M., Qasse, I., Liebel, G., Hamdaqa, M., 2023. Automesc: Automatic framework for mining and classifying ethereum smart contract vulnerabilities and their fixes. In: 2023 49th Euromicro Conference on Software Engineering and Advanced Applications. SEAA, IEEE, pp. 410–417.
- Sun, X., Tu, L., Zhang, J., Cai, J., Li, B., Wang, Y., 2023. Assbert: Active and semi-supervised bert for smart contract vulnerability detection. J. Inf. Secur. Appl. 73, 103423.
- Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y., 2024. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. pp. 1–13.
- Szabo, N., 1996. Smart contracts: building blocks for digital markets. EXTROPY: J. Transhumanist Thought 18 (16), 28.
- Team, V., 2025. Principles and goals. URL: <https://docs.vyperlang.org/en/stable/>.
- Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., Alexandrov, Y., 2018a. Smartcheck: Static analysis of ethereum smart contracts. In: 2018 IEEE/ACM 1st International Workshop on Emerging Trends in Software Engineering for Blockchain. WETSEB, IEEE, pp. 9–16.
- Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., Alexandrov, Y., 2018b. Smartcheck: Static analysis of ethereum smart contracts. In: Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain. pp. 9–16.
- Torres, C.F., Schütte, J., et al., 2018. Osiris: Hunting for integer bugs in ethereum smart contracts. In: Proceedings of the 34th Annual Computer Security Applications Conference. ACM, pp. 664–676.
- Vitalik Buterin, G.W., 2014. Archived ethereum white paper. https://static.peng37.com/ethereum_whitepaper_laptop_3.pdf.
- Wartschinski, L., Noller, Y., Vogel, T., Kehrer, T., Grunske, L., 2022. Vudenc: vulnerability detection with deep learning on a natural codebase for python. Inf. Softw. Technol. 144, 106809.
- Weiser, M., 1984. Program slicing. IEEE Trans. Softw. Eng. 352–357.
- Wood, G., et al., 2014. Ethereum: A secure decentralised generalised transaction ledger. Ethereum Proj. Yellow Pap. 151, 1–32.
- Zhang, P., Xiao, F., Luo, X., 2020. A framework and dataset for bugs in ethereum smart contracts. In: 2020 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, pp. 139–150.
- Zhang, Z., Zhang, B., Xu, W., Lin, Z., 2023. Demystifying exploitable bugs in smart contracts. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 615–627.
- Zhao, J., Chen, X., Yang, G., Shen, Y., 2024. Automatic smart contract comment generation via large language models and in-context learning. Inf. Softw. Technol. 168, 107405.
- Zheng, Z., Su, J., Chen, J., Lo, D., Zhong, Z., Ye, M., 2024. Dappscan: building large-scale datasets for smart contract weaknesses in dapp projects. IEEE Trans. Softw. Eng..
- Zheng, Z., Xie, S., Dai, H.-N., Chen, X., Wang, H., 2018. Blockchain challenges and opportunities: A survey. Int. J. Web Grid Serv. 14, 352–375.
- Zhou, Y., Sharma, A., 2017. Automated identification of security issues from commit messages and bug reports. In: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. pp. 914–919.